



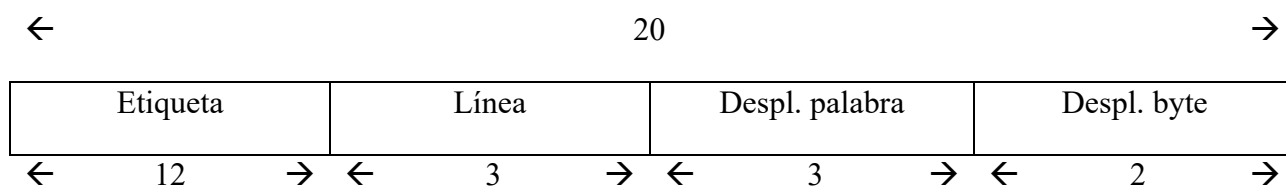
HOJA DE PROBLEMAS 4: JERARQUÍA DE MEMORIA (PARTE 1)

1. Sea una memoria caché de correspondencia directa. Suponiendo siempre que la unidad direccionable de la memoria es el byte, realizar la división en campos de una dirección de memoria en los siguientes casos:

Apartado	Tamaño de la dirección de memoria	Número de líneas	Tamaño de bloque	Tamaño de palabra
a)	20 bits	8	8 palabras	32 bits
b)	32 bits	128	64 palabras	64 bits
c)	16 bits	32	2 palabras	8 bits
d)	36 bits	1024	128 palabras	16 bits

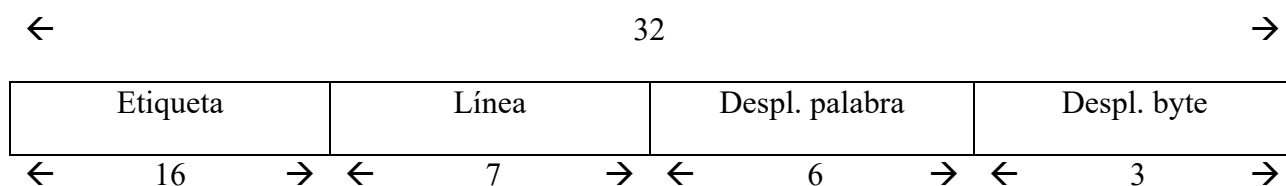
Solución:

- a) Las direcciones de memoria son de 20 bits y se descomponen de la siguiente manera:



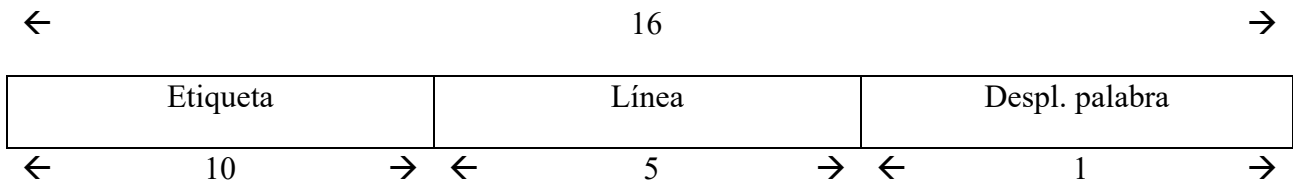
- El desplazamiento de byte es de 2 bits porque las palabras son de 32 bits = 4 bytes = 2^2 bytes.
- El desplazamiento de palabra es de 3 bits porque los bloques son de 8 palabras = 2^3 palabras.
- El número de línea se expresa con 3 bits porque hay 8 líneas = 2^3 líneas.
- El resto de los bits hasta completar los 20 de la dirección se dedican a la etiqueta, en concreto, $20 - 3 - 3 - 2 = 12$ bits.

- b) Las direcciones de memoria son de 32 bits y se descomponen de la siguiente manera:



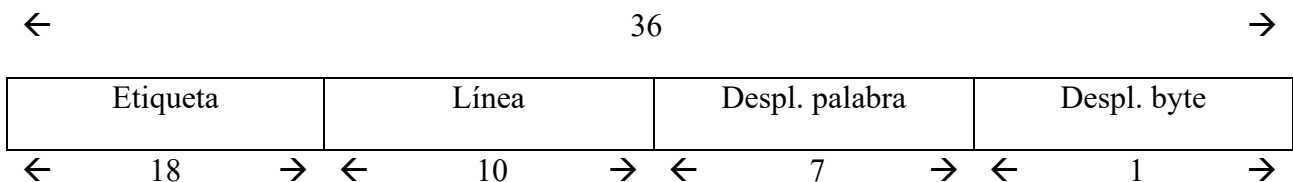
- El desplazamiento de byte es de 3 bits porque las palabras son de 64 bits = 8 bytes = 2^3 bytes.
- El desplazamiento de palabra es de 6 bits porque los bloques son de 64 palabras = 2^6 palabras.
- El número de línea se expresa con 7 bits porque hay 128 líneas = 2^7 líneas.
- El resto de los bits hasta completar los 32 de la dirección se dedican a la etiqueta, en concreto, $32 - 7 - 6 - 3 = 16$ bits.

c) Las direcciones de memoria son de 16 bits y se descomponen de la siguiente manera:



- El desplazamiento de byte es de 0 bits porque las palabras son de 8 bits = 1 byte = 2^0 bytes.
- El desplazamiento de palabra es de 1 bit porque los bloques son de 2 palabras = 2^1 palabras.
- El número de línea se expresa con 5 bits porque hay 32 líneas = 2^5 líneas.
- El resto de los bits hasta completar los 16 de la dirección se dedican a la etiqueta, en concreto, $16 - 5 - 1 - 0 = 10$ bits.

d) Las direcciones de memoria son de 36 bits y se descomponen de la siguiente manera:



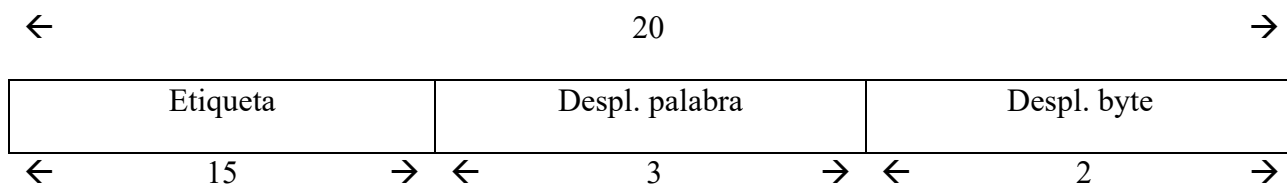
- El desplazamiento de byte es de 1 bit porque las palabras son de 16 bits = 2 bytes = 2^1 bytes.
- El desplazamiento de palabra es de 7 bits porque los bloques son de 128 palabras = 2^7 palabras.
- El número de línea se expresa con 10 bits porque hay 1024 líneas = 2^{10} líneas.
- El resto de los bits hasta completar los 36 de la dirección se dedican a la etiqueta, en concreto, $36 - 10 - 7 - 1 = 18$ bits.

2. Sea una memoria caché de correspondencia totalmente asociativa. Suponiendo siempre que la unidad direccionable de la memoria es el byte, realizar la división en campos de una dirección de memoria en los siguientes casos:

Apartado	Tamaño de la dirección de memoria	Número de líneas	Tamaño de bloque	Tamaño de palabra
a)	20 bits	8	8 palabras	32 bits
b)	32 bits	128	64 palabras	64 bits
c)	16 bits	32	2 palabras	8 bits
d)	36 bits	1024	128 palabras	16 bits

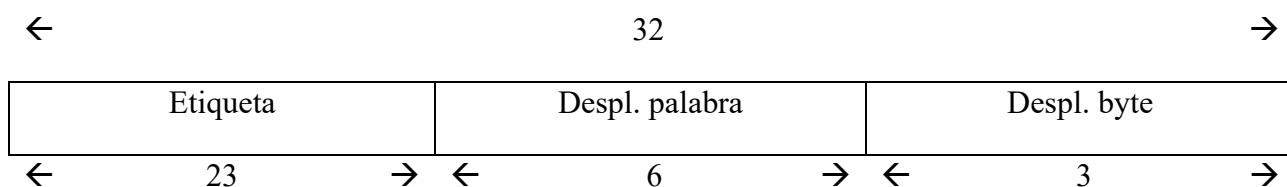
Solución:

- a) Las direcciones de memoria son de 20 bits y se descomponen de la siguiente manera:



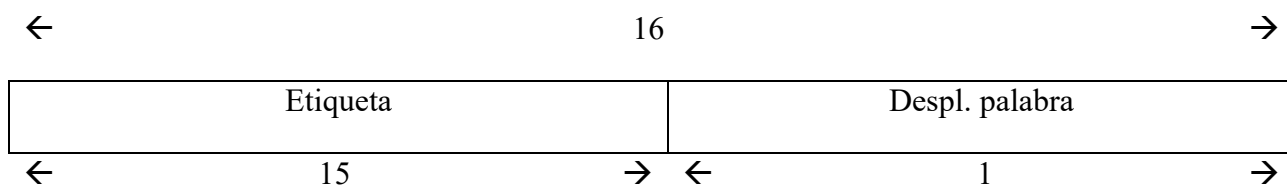
- El desplazamiento de byte es de 2 bits porque las palabras son de 32 bits = 4 bytes = 2^2 bytes.
- El desplazamiento de palabra es de 3 bits porque los bloques son de 8 palabras = 2^3 palabras.
- El resto de los bits hasta completar los 20 de la dirección se dedican a la etiqueta, en concreto, $20 - 3 - 2 = 15$ bits.

- b) Las direcciones de memoria son de 32 bits y se descomponen de la siguiente manera:



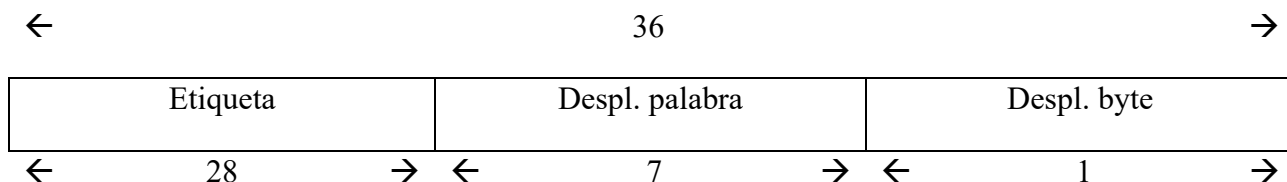
- El desplazamiento de byte es de 3 bits porque las palabras son de 64 bits = 8 bytes = 2^3 bytes.
- El desplazamiento de palabra es de 6 bits porque los bloques son de 64 palabras = 2^6 palabras.
- El resto de los bits hasta completar los 32 de la dirección se dedican a la etiqueta, en concreto, $32 - 6 - 3 = 23$ bits.

c) Las direcciones de memoria son de 16 bits y se descomponen de la siguiente manera:



- El desplazamiento de byte es de 0 bits porque las palabras son de 8 bits = 1 byte = 2^0 bytes.
- El desplazamiento de palabra es de 1 bit porque los bloques son de 2 palabras = 2^1 palabras.
- El resto de los bits hasta completar los 16 de la dirección se dedican a la etiqueta, en concreto, $16 - 1 - 0 = 15$ bits.

d) Las direcciones de memoria son de 36 bits y se descomponen de la siguiente manera:



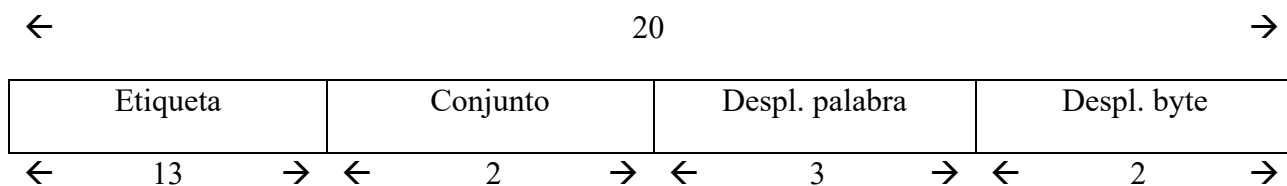
- El desplazamiento de byte es de 1 bit porque las palabras son de 16 bits = 2 bytes = 2^1 bytes.
- El desplazamiento de palabra es de 7 bits porque los bloques son de 128 palabras = 2^7 palabras.
- El resto de los bits hasta completar los 36 de la dirección se dedican a la etiqueta, en concreto, $36 - 7 - 1 = 28$ bits.

3. Sea una memoria caché de correspondencia asociativa por conjuntos. Suponiendo siempre que la unidad direccionable de la memoria es el byte, realizar la división en campos de una dirección de memoria en los siguientes casos:

Apartado	Tamaño de la dirección de memoria	Número de conjuntos	Número de vías	Tamaño de bloque	Tamaño de palabra
a)	20 bits	4	2	8 palabras	32 bits
b)	32 bits	16	8	64 palabras	64 bits
c)	16 bits	8	4	2 palabras	8 bits
d)	36 bits	64	16	128 palabras	16 bits

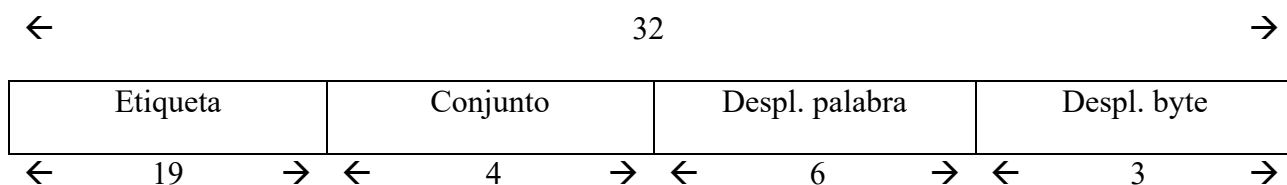
Solución:

a) Las direcciones de memoria son de 20 bits y se descomponen de la siguiente manera:



- El desplazamiento de byte es de 2 bits porque las palabras son de 32 bits = 4 bytes = 2^2 bytes.
- El desplazamiento de palabra es de 3 bits porque los bloques son de 8 palabras = 2^3 palabras.
- El número de conjunto se expresa con 2 bits porque hay 4 conjuntos = 2^2 conjuntos.
- El resto de los bits hasta completar los 20 de la dirección se dedican a la etiqueta, en concreto, $20 - 2 - 3 - 2 = 13$ bits.

b) Las direcciones de memoria son de 32 bits y se descomponen de la siguiente manera:



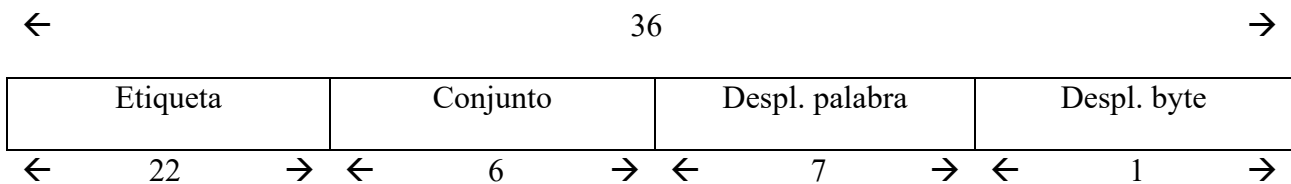
- El desplazamiento de byte es de 3 bits porque las palabras son de 64 bits = 8 bytes = 2^3 bytes.
- El desplazamiento de palabra es de 6 bits porque los bloques son de 64 palabras = 2^6 palabras.
- El número de conjunto se expresa con 4 bits porque hay 16 conjuntos = 2^4 conjuntos.
- El resto de los bits hasta completar los 32 de la dirección se dedican a la etiqueta, en concreto, $32 - 4 - 6 - 3 = 19$ bits.

c) Las direcciones de memoria son de 16 bits y se descomponen de la siguiente manera:



- El desplazamiento de byte es de 0 bits porque las palabras son de 8 bits = 1 byte = 2^0 bytes.
- El desplazamiento de palabra es de 1 bit porque los bloques son de 2 palabras = 2^1 palabras.
- El número de conjunto se expresa con 3 bits porque hay 8 conjuntos = 2^3 conjuntos.
- El resto de los bits hasta completar los 16 de la dirección se dedican a la etiqueta, en concreto, $16 - 3 - 1 - 0 = 12$ bits.

d) Las direcciones de memoria son de 36 bits y se descomponen de la siguiente manera:



- El desplazamiento de byte es de 1 bit porque las palabras son de 16 bits = 2 bytes = 2^1 bytes.
- El desplazamiento de palabra es de 7 bits porque los bloques son de 128 palabras = 2^7 palabras.
- El número de conjunto se expresa con 6 bits porque hay 64 conjuntos = 2^6 conjuntos.
- El resto de los bits hasta completar los 36 de la dirección se dedican a la etiqueta, en concreto, $36 - 6 - 7 - 1 = 22$ bits.

4. **[Ejercicio de examen]** Tres computadores disponen del mismo procesador, que tiene un bus de datos de 32 bits y un bus de direcciones también de 32 bits. Cada uno de ellos tiene una memoria caché con capacidad para 4 kB (sin contar etiquetas ni bits de control) y todos ellos realizan transferencias entre caché y memoria principal mediante bloques de 4 palabras.

- El primer computador posee de una memoria caché de correspondencia directa.
- El segundo computador tiene una caché totalmente asociativa.
- El tercer computador tiene una caché asociativa por conjuntos de 8 vías.

Se pide indicar en qué campos se divide una dirección de memoria, en qué orden se encuentra cada uno, así como su tamaño en bits. En los tres casos habrá que realizar un dibujo explicativo de la división en campos de la dirección.

- d) Si el tiempo de acceso al nivel superior de la jerarquía de memoria es de 2,4 ns y al inferior es de 40 ns, ¿cuál debe ser la tasa de aciertos de la caché para conseguir un tiempo medio de acceso a la memoria de 5 ns?

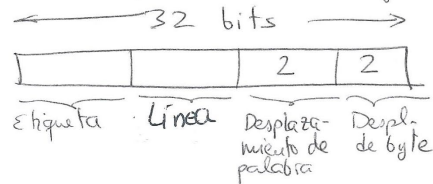
Solución: [https://youtu.be/ bGle7S2Gy4](https://youtu.be/bGle7S2Gy4)

Procesador $\left\{ \begin{array}{l} \text{Bus de datos: } 32 \text{ bits} = 4 \text{ bytes} = \text{Tamaño de palabra} \\ \text{Bus de direcciones: } 32 \text{ bits.} \end{array} \right.$
 $\begin{array}{l} \downarrow \\ 2 \text{ bits} \end{array}$

Caché : Capacidad = 4 KB (solo datos).

↳ Tamaño de bloque = 4 palabras = $2^2 \rightarrow 2 \text{ bits}$

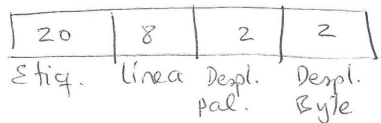
a) Correspondencia directa: Un bloque de memoria va siempre a la misma línea de caché.
 La dirección se descompone en los siguientes campos:



Tenemos que averiguar cuántas líneas de caché hay. Cada línea tiene un bloque: $4 \text{ palabras} \times 4 \frac{\text{bytes}}{\text{pal}} = 16 \text{ bytes}$

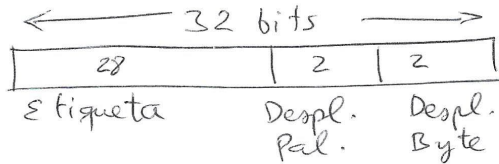
$$N^{\circ} \text{ líneas} = \frac{4 \text{ KB}}{16 \text{ bytes/línea}} = \frac{2^{12}}{2^4} = 2^8 \text{ líneas} = 256 \text{ líneas} \Rightarrow 8 \text{ bits}$$

luego el tamaño de la etiqueta es $32 - 8 - 2 - 2 = \boxed{20 \text{ bits}}$



b) Cache totalmente asociativa: un bloque de memoria va a cualquier línea de caché.

En este caso la dirección se descompone en:

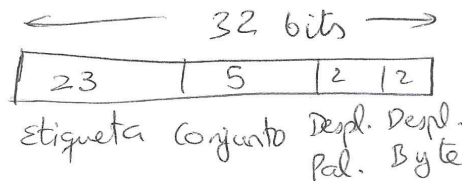


La etiqueta es de tamaño $32 - 2 - 2 = 28$ bits

c) Cache asociativa por conjuntos de 8 vías: un bloque de memoria va siempre al mismo conjunto, pero podemos elegir entre cualquiera de las 8 vías del mismo.

$$\text{Nº conjuntos} = \frac{4 \text{ KB}}{4 \frac{\text{pal}}{\text{vía}} \times 4 \frac{\text{bytes}}{\text{pal}} \times 8 \text{ vías/conj}} = \frac{2^{12}}{2^2 \cdot 2^2 \cdot 2^3} = 2^5 = 32 \text{ conjuntos (5 bits)}$$

$$\text{Etiqueta: } 32 - 5 - 2 - 2 = 23 \text{ bits}$$



d) El tiempo medio de acceso a la memoria (AMAT) se puede escribir como:

$$AMAT = T_{\text{acierto}} + \% \text{ fallo} \times T_{\text{penalización}}$$

Donde:

- $AMAT = 5 \text{ ns}$
- T_{acierto} (tiempo de acceso a la caché) = 2,4 ns
- $T_{\text{penalización}}$ (tiempo de acceso a la memoria principal) = 40 ns

Luego:

$$\% \text{ fallo} = \frac{AMAT - T_{\text{acierto}}}{T_{\text{penalización}}} = \frac{5 \text{ ns} - 2,4 \text{ ns}}{40 \text{ ns}} = 0,065 = 6,5 \%$$

Por tanto:

$$\% \text{ acierto} = 100 - \% \text{ fallo} = 100 - 6,5 = 93,5 \%$$

5. [Ejercicio de examen] Sea un computador dotado con una jerarquía de memoria que incluye memoria caché y memoria principal. El ancho de palabra es de 32 bits y las direcciones de memoria son de 26 bits, con unidad direccionable el byte. Los detalles de la memoria caché son los siguientes:

- La caché es de correspondencia directa con 4 líneas.
- La transferencia entre la memoria principal y la caché se realiza mediante bloques de 16 palabras.
- La política de escritura es *write-back* y *write-allocate*.
- Aparte de etiquetas y datos, la caché posee para cada línea un bit de validez (V) y un bit de suciedad (D).
- El contenido parcial de la caché en un momento de dado es el siguiente:

Línea	Etiqueta	V	D	Bloque de datos
0	39F0E	0	0	...
1	1D728	1	1	...
2	2A584	1	1	...
3	123E5	1	0	...

(etiquetas en hexadecimal, número de línea en decimal, por simplicidad no se muestran los datos)

Se pide:

- Indicar detalladamente los campos en los que se divide una dirección de memoria.
- Durante la ejecución de un programa se realiza la siguiente secuencia ordenada de accesos a memoria (direcciones en hexadecimal). Considerar que cada cambio en la caché afecta a los accesos siguientes. Indicar detalladamente si son aciertos o fallos de caché, explicar detalladamente cómo va cambiando la caché a lo largo del tiempo, si se actualiza la memoria principal y cuál es el estado final de la memoria caché.

- 0x2E16D29 (lectura)
- 0x340C656 (escritura)
- 0x123E5D9 (escritura)
- 0x2A584B4 (lectura)

Solución:

- Las direcciones de memoria son de 26 bits y se descomponen de la siguiente manera:

← 26 →

Etiqueta	Línea	Despl. palabra	Despl. byte
← 18 →	← 2 →	← 4 →	← 2 →

- El desplazamiento de byte es de 2 bits porque las palabras son de 32 bits = 4 bytes = 2^2 bytes.
- El desplazamiento de palabra es de 4 bits porque los bloques son de 16 palabras = 2^4 palabras.
- El número de línea se expresa con 2 bits porque hay 4 líneas = 2^2 líneas.
- El resto de los bits hasta completar los 26 se dedican a la etiqueta, en concreto, $26 - 2 - 4 - 2 = 18$ bits.

b) Veamos ahora qué pasa con cada uno de los accesos a memoria:

b.1) 0x2E16D29 (lectura)

Descomposición en campos:

0x2E16D29 = 10 1110 0001 0110 1101 0010 1001 = [10 1110 0001 0110 1101]
[00] [1010] [01]

DB = 01

DP = 1010

Línea = 00

Etiqueta = 10 1110 0001 0110 1101 = 0x2E16D

Vamos a la línea 0 y vemos que V=0, con lo que es un dato no válido, con lo que es un FALLO. Podemos sustituir este bloque directamente. Por tanto, traemos el bloque de datos correspondiente a esta dirección desde la memoria principal hasta la línea 0 y activamos V. También actualizamos la etiqueta. La caché ahora tiene este contenido (remarcados en rojo los cambios):

Línea	Etiqueta	V	D	Bloque de datos
0	2E16D	1	0	...
1	1D728	1	1	...
2	2A584	1	1	...
3	123E5	1	0	...

b.2) 0x340C656 (escritura)

Descomposición en campos:

0x340C656 = 11 0100 0000 1100 0110 0101 0110 = [11 0100 0000 1100 0110]
[01] [0101] [10]

DB = 10

DP = 0101

Línea = 01

Etiqueta = 11 0100 0000 1100 0110 = 0x340C6

Vamos a la línea 1 y vemos que V=1, pero no hay coincidencia de etiquetas, con lo que es un FALLO. Antes de sustituir este bloque, y debido a que D=1, debemos copiarlo desde la memoria caché hacia la memoria principal. A continuación, por ser *write-allocate*, traemos el nuevo bloque desde la memoria principal hasta la línea 1 de la caché. Y por ser *write-back*, modificamos el dato en el bloque de la memoria caché, con lo que D sigue valiendo 1. El bloque de memoria principal pasa a estar anticuado.

Por tanto, el estado de la caché es el siguiente:

Línea	Etiqueta	V	D	Bloque de datos
-------	----------	---	---	-----------------

0	2E16D	1	0	...
1	340C6	1	1	...
2	2A584	1	1	...
3	123E5	1	0	...

b.3) 0x123E5D9 (escritura)

Descomposición en campos:

0x123E5D9 = 01 0010 0011 1110 0101 1101 1001 = [01 0010 0011 1110 0101]
[11] [0110] [01]

DB = 01

DP = 0110

Línea = 11

Etiqueta = 01 0010 0011 1110 0101 = 0x123E5

Vamos a la línea 3 y observamos que tanto V=1 como que hay coincidencia de etiquetas. Por tanto, es un ACIERTO. Al tratarse de *write-back*, realizamos la escritura solo en la caché y, por ello, activamos D=1. La caché queda así:

Línea	Etiqueta	V	D	Bloque de datos
0	2E16D	1	0	...
1	340C6	1	1	...
2	2A584	1	1	...
3	123E5	1	1	...

b.4) 0x2A584B4 (lectura)

Descomposición en campos:

0x2A584B4 = 10 1010 0101 1000 0100 1011 0100 = [10 1010 0101 1000 0100]
[10] [1101] [00]

DB = 00

DP = 1101

Línea = 10

Etiqueta = 10 1010 0101 1000 0100 = 0x2A584

Vamos a la línea 2 y observamos que tanto V=1 como que hay coincidencia de etiquetas. Por tanto, es un ACIERTO. Al tratarse de una lectura, leemos el dato solicitado directamente de la memoria caché y no es necesario realizar ningún cambio.

El estado final de la caché es el siguiente:

Línea	Etiqueta	V	D	Bloque de datos
0	2E16D	1	0	...
1	340C6	1	1	...
2	2A584	1	1	...
3	123E5	1	1	...

6. **[Ejercicio de examen]** Sea un computador con direcciones de memoria de 32 bits y palabras de tamaño 4 bytes, con unidad direccionable el byte. Dispone de una memoria caché de correspondencia directa con 8 líneas, que realiza intercambios con la memoria principal mediante bloques de 16 palabras. La política de escritura es *write-through*. Se pide:

- Descomponer la dirección de memoria en campos según su formato y explicar su significado.
- Calcular el tamaño total de la memoria caché, incluida la parte de los bloques de datos.
- Suponiendo que durante la ejecución de un programa el estado de la caché es el que se muestra en la tabla inferior, explicar cómo va cambiando su contenido y si se producen aciertos o fallos tras la secuencia de lecturas a memoria con las direcciones siguientes (expresadas en hexadecimal). Dibujar también el estado final de la caché tras las cuatro lecturas:

c.1) 0x64AF246B

c.2) 0XFF52A906

c.3) 0xB5BF51D1

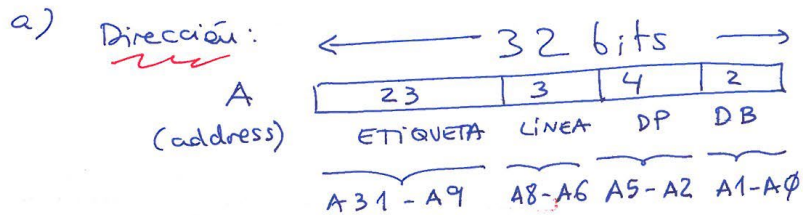
c.4) 0x052ADD00

Los efectos de una dirección de memoria se han de tener en cuenta para los accesos posteriores. El estado de la caché inmediatamente antes del apartado **c.1)** es el siguiente:

Línea	Etiqueta	V	Bloque de datos
0	0x7BA091	0	...
1	0x325792	0	...
2	0x030144	1	...
3	0x592345	1	...
4	0x20FEDC	1	...
5	0x6D4639	1	...
6	0x1F109C	0	...
7	0x5ADFA8	1	...

Solución: https://youtu.be/y8_Xo0ScuKQ

Direcciones de memoria = 32 bits
 Palabras = 4 bytes = 2^2 B $\Rightarrow$ DB = 2 bits
 Bloques = 16 palabras = 2^4 pal $\Rightarrow$ DP = 4 bits.
 Caché correspondencia directa $\rightarrow$ 8 líneas $\rightarrow$ 3 bits
 $\hookrightarrow$ Write-through 2^3 índice de línea



b) Tamaño de la caché:

* El número de posiciones (líneas) de caché es 8 (tamaño vertical).

* En horizontal tenemos:

- 67 bytes

- 1 bit de validez (V).
 - 1 etiqueta de 23 bits.
 - 1 bloque de 16 palabras, cada una de 4 bytes = 64 bytes
 - Al ser la política de escritura write-through, no hay bit de suciedad, ya que caché y MP contienen el mismo dato.

}

24 bits = 3 bytes

Tamaño total $\Rightarrow 8 \text{ líneas} \times 67 \text{ bytes/línea} = \boxed{536 \text{ bytes}}$

c)

c.1) $0 \times 64AF246B = \underline{0110 \ 0100 \ 1010 \ 1111 \ 0010}$
 $\underline{0100 \ 0110 \ 1011}$
 ETIQ. LÍNEA DP DB

$\hookrightarrow \text{Etiqueta} = \frac{011}{3} \frac{0010}{2} \frac{0101}{5} \frac{0111}{7} \frac{1001}{9} \frac{0010}{2}$

Línea = $001 = 1_{10}$

DP = $1010 = 10_{10}$

DB = $11 = 3_{10}$

Vamos a línea 1 y hay coincidencia de etiquetas, pero el bit de validez está desactivado. $\Rightarrow$ FALLO.

Como única acción, se activa V a 1. La etiqueta no hace falta cambiarla y lo único que haríamos sería actualizar el bloque de datos.

c.2) $0 \times FF52A906 = \underline{1111 \ 1111 \ 0101 \ 0010 \ 1010}$
 $\underline{1001 \ 0000 \ 0110}$
 ETIQ. LÍNEA DP DB

$\hookrightarrow \text{Etiqueta} = \frac{111}{7} \frac{1111}{F} \frac{1010}{A} \frac{1001}{9} \frac{0101}{5} \frac{0100}{4}$

Línea = $100 = 4_{10}$

DP = $0001 = 1_{10}$

DB = $10 = 2_{10}$

Vamos a línea 4. Las etiquetas no coinciden $\Rightarrow$ FALLO. Se actualiza la etiqueta y el bloque de datos. El bit V se mantiene a 1.

$$c.3) \quad 0x B5BF51D1 = \underbrace{1011 \ 0101 \ 1011 \ 1111 \ 0101}_{\text{ETIQA.}}$$

$$\underbrace{0001 \ 1101 \ 0001}_{\text{ETIQA. LINEA DP DB}}$$

$$\rightarrow \text{Etiqueta} = \underbrace{101}_5 \ \underbrace{1010}_A \ \underbrace{1101}_D \ \underbrace{1111}_F \ \underbrace{1010}_A \ \underbrace{1000}_8$$

$$\text{Linea} = 111 = 7_{10}$$

$$\text{DP} = 0100 = 4_{10}$$

$$\text{DB} = 01 = 1_{10}$$

Vamos a línea 7. Hay coincidencia de etiquetas y además $V=1 \Rightarrow$ ACIERTO.

$$c.4) \quad 0x 052 \text{ ADD } 00 = \underbrace{0000 \ 0101 \ 0010 \ 1010 \ 1101}_{\text{ETIQA.}}$$

$$\underbrace{1101 \ 0000 \ 0000}_{\text{ETIQA. LINEA DP DB}}$$

$$\rightarrow \text{Etiqueta} = \underbrace{000}_0 \ \underbrace{0010}_2 \ \underbrace{1001}_9 \ \underbrace{0101}_5 \ \underbrace{0110}_6 \ \underbrace{1110}_E$$

$$\text{Linea} = 100 = 4_{10}$$

$$\text{DP} = 0000 = 0_{10}$$

$$\text{DB} = 00 = 0_{10}$$

Vamos a línea 4. las etiquetas no coinciden.

⇒ FALLO. Se actualiza la etiqueta y el bloque de datos. El bit de validat se mantiene a 1.

El estado final de la caché es el siguiente:

Línea	V	Etiqueta
0	0	0x7BA091
1	0 , 1	0x325792
2	1	0x030144
3	1	0x592345
4	1	0x20FEDC , 0x7FA954 , 0x02956E
5	1	0x6D4639
6	0	0x1F109C
7	1	0x5ADFA8

Se muestran con valores tachados los que han sido borrados de la caché.

7. [Ejercicio de examen] Sea un computador basado en el procesador MIPS, con una memoria caché de datos totalmente asociativa, de tamaño 32 bytes, con bloques de 2 palabras. El tamaño de palabra y el de la dirección de memoria es el habitual (32 bits). La política de reemplazo de la caché es LRU. La unidad direccionable es el byte, si bien la memoria permite accesos con tamaño palabra, media palabra o byte. La ordenación de los bytes de una palabra es *little-endian*. Se pide:

- a) Escribir la descomposición en campos de una dirección de memoria desde el punto de vista de la caché.
- b) Antes de la ejecución del programa, la caché se encuentra vacía. Suponiendo que el registro **\$s0** contiene el valor **0x10010000**, indicar qué efectos producen las siguientes instrucciones de carga desde el punto de vista de la caché (dirección de memoria leída, valor de etiqueta, acierto/fallo, número de bloque de caché implicado en la operación), así como sobre el valor del registro **\$s1**. Rellenar la tabla que se adjunta a modo de resumen.

- b.1) **lw** **\$s1**, 0(**\$s0**)
- b.2) **lh** **\$s1**, 4(**\$s0**)
- b.3) **lb** **\$s1**, 1(**\$s0**)
- b.4) **lw** **\$s1**, 8(**\$s0**)
- b.5) **lw** **\$s1**, 16(**\$s0**)
- b.6) **lh** **\$s1**, 14(**\$s0**)

IMPORTANTE: Cada acceso a memoria afecta a los apartados posteriores.

Contenido de la memoria al comienzo del programa:

Data Segment					
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)
0x10010000	0xaf320910	0xffe09863	0xabcdef01	0x12345678	0x55667788

Recordatorio de instrucciones:

- **lw** = carga de palabra
- **lh** = carga de media palabra (se guarda en la media palabra inferior de un registro, con extensión de signo del bit 15)
- **lb** = carga de byte (se guarda en el byte inferior de un registro, con extensión de signo del bit 7)
- El primer operando es el registro destino de la operación
- El segundo operando indica la fuente del dato en memoria, mediante un direccionamiento indirecto a registro con desplazamiento (valor del registro + desplazamiento numérico)

TABLA DE RESUMEN DE RESULTADOS:

Apartado	Dirección leída (hexadecimal)	Etiqueta (hexadecimal)	Acierto/Fallo	Bloque	Registro \$s1 después de instrucción (hexadecimal)
b.1)					
b.2)					
b.3)					
b.4)					
b.5)					
b.6)					

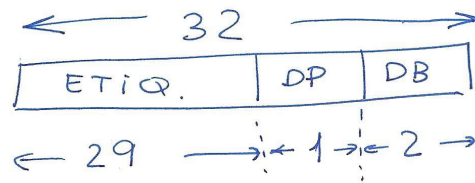
Solución: <https://youtu.be/8W0ADMBp1DU>

Datos del problema:

- * Tamaño de la caché = 32 bytes (caché de juguete)
- * Totalmente asociativa
- * 2 palabras/bloque = $2^1 \Rightarrow q = 1$
- * 32 bits/palabra = 4 bytes/palabra = $2^2 \Rightarrow q = 2$
- * Tamaño de dirección de memoria = 32 bits
- * Reemplazo LRU (usado menos recientemente)
- * Unidad direccionable = byte
 - ↳ Accesos permitidos
 - Byte → cualquier dirección
 - Half (2 B) → dirección par
 - Word (4 B) → dirección múltiplo de 4
- * Orden little-endian
 - ↳ En una palabra (o en una media palabra) el byte menos significativo se encuentra en la dirección de memoria más baja.

$$\begin{aligned} \text{N.º de líneas de caché} &= \frac{32 \text{ B}}{2 \text{ palabras/línea} \times 4 \text{ B/palabra}} = 4 \text{ líneas} \end{aligned}$$

a) Descomposición de la dirección de memoria en campos:



- DB = desplazamiento de byte → esta parte de la dirección identifica un byte de los 4 que constituyen una palabra.
- DP = desplazamiento de palabra → esta parte de la dirección identifica una palabra de las 2 que tiene un bloque.
- ETIQUETA = los bits superiores de la dirección sirven para identificar un bloque de memoria principal en la caché y diferenciarlo de los demás.

b) El registro base $\$5\phi$ vale $0x10010000$.

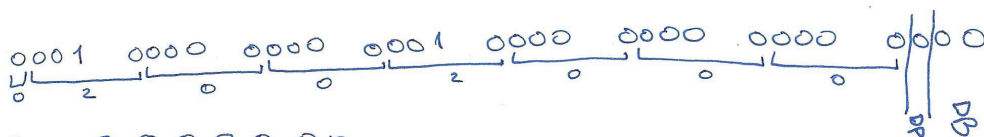
b.1) $lw \$51, \phi(\$5\phi)$

→ Carga una palabra (4B) en $\$51$.

Dirección de memoria:

Base: $0x10010000$

Desplazamiento $^0 +$
 $\hline 0x10010000$



ETIQU: $0x02002000$

Como la caché está vacía, es un fallo (el forzoso).
 Suponiendo que la línea menos reciente es la ϕ , asignamos a esa línea el bloque leído.

La palabra contenida en la dirección $0x10010000$ es $0x AF32 0910$. Nótese que el bloque leído son esta palabra y la siguiente ($0xFFE09863$).

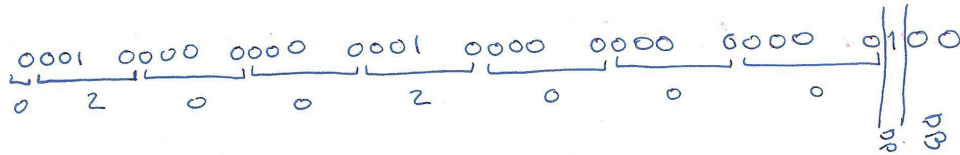
$\$s1 \leftarrow 0x AF32 0910$. ↖ en la dirección $0x10010004$

b.2) lh $\$s1, 4(\$s0)$ → Carga media palabra (2B) en $\$s1$. El bit 15 de la media palabra leída tiene extensión de signo del bit 16 al 31.

Dirección de memoria:

Base: $0x10010000$

Despl.: $\begin{array}{r} 4 \\ + \\ \hline 0x10010004 \end{array}$



ETIQ: $0x02002000$

Hay ACIERTO, porque esta etiqueta está en la línea 0.

Contenido → FF E0 98 63 !! Little-endian!!
 Dirección → 10010007 10010006 10010005 10010004

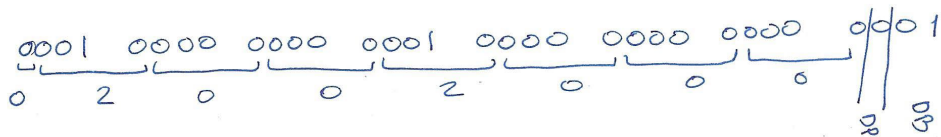
La media palabra en la dirección $0x10010004$ se compone de los bytes $0x9863$. Como hay extensión de signo
 $\$s1 \leftarrow 0x FFFF9863$

b.3) $lb \$s1, 1(\$5\phi)$

Carga 1 byte en $\$s1$.
El bit 7 del byte leído sufre
extensión de signo del bit
8 al 31.

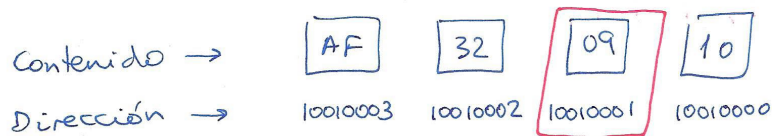
Dirección de memoria:

$$\begin{array}{r} \text{Base: } 0x10010000 \\ \text{Despl: } \quad \quad \quad 1 \quad + \\ \hline 0x10010001 \end{array}$$



ETIQ. $0x02002000$

Vuelve a ser ACIERTO, ya que el bloque buscado está
en la línea ϕ .



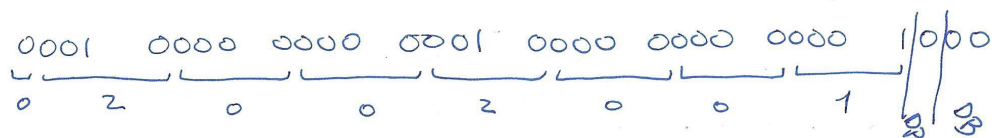
El byte en la dirección $0x10010001$ es $0x09$. Luego
 $\$s1 \leftarrow 0x00000009$

b.4) $lw \$s1, 8(\$5\phi)$

Carga una palabra en $\$s1$

Dirección de memoria:

$$\begin{array}{r} \text{Base: } 0x10010000 \\ \text{Despl: } \quad \quad \quad 8 \quad + \\ \hline 0x10010008 \end{array}$$



ETIQ. $0x02002001$

Esta etiqueta no se encuentra en ninguna línea de caché $\Rightarrow$ FALLO. El bloque leído lo asignamos a la línea menos reciente $\Rightarrow$ La 1.

$\$51 \leftarrow 0xABCDEF01$

b.5) $lw \$51, 16(\$50)$ Carga una palabra en $\$51$.

Dirección de memoria:

Base: $0x10010000$

16 en hexadecimal es $0x10$.

Despl: $0x10 +$

$0x10010010$

0001	0000	0000	0001	0000	0000	0001	0000
0	2	0	0	2	0	0	2
							4 0B

ETIQ. $0x02002002$

Esta etiqueta no se encuentra en ninguna línea de caché.
 $\Rightarrow$ FALLO. El bloque leído lo asignamos a la línea menos reciente $\Rightarrow$ La 2.

$\$51 \leftarrow 0x55667788$

b.6) lh \$s1, 14 (\$s0)

Dirección de memoria:

Base: 0x10010000

Despl 0xE +

 0x1001000E

Carga media palabra en \$s1. El bit 15 de la media palabra leída tiene extensión de signo del bit 16 al 31.

14 en hexadecimal es 0xE.

0001 0000 0000 0001 0000 0000 0000 1110
 0 2 0 0 2 0 0 1 4p DB

Etiqu : 0x02002001

⇒ ACERTO. Este bloque se encuentra en la línea 1.

Contenido →

12	34	56	78
----	----	----	----

 Dirección →

1001000F	1001000E	1001000D	1001000C
----------	----------	----------	----------

la media palabra en la dirección 0x1001000E se compone de los bytes 0x1234. luego \$s1 ← 0x00001234

RESUMEN:

Apartado	Dirección	Etiqueta	A/F	Bloque	\$s1
b.1)	0x10010000	0x02002000	F	0	0xAF320910
b.2)	0x10010004	0x02002000	A	0	0xFFFF9863
b.3)	0x10010001	0x02002000	A	0	0x00000009
b.4)	0x10010008	0x02002001	F	1	0xABCDEF01
b.5)	0x10010010	0x02002002	F	2	0x55667788
b.6)	0x1001000E	0x02002001	A	1	0x00001234

8. **[Ejercicio de examen]** Sea un computador con direcciones de memoria de 36 bits y palabras de tamaño 64 bits, con unidad direccionable el byte. Dispone de una memoria caché asociativa por conjuntos de 2 vías, con una capacidad total de 8 kB, que realiza intercambios con la memoria principal mediante bloques de 64 palabras. Se pide:

- Descomponer la dirección de memoria en campos según su formato y explicar su significado.
- Suponiendo que la caché está inicialmente vacía, dibujar la caché mediante una tabla en la que se indique cómo va cambiando su contenido y si se producen aciertos o fallos tras la secuencia de lecturas a memoria con las direcciones siguientes (expresadas en hexadecimal):

- 0xABCDEF012
- 0x803223106
- 0x250917E83
- 0x1234570ED
- 0xEDCEFD393
- 0x01630DF36

Considerar para el dibujo solamente para cada vía: el bit de validez V, el bit de reciente uso R y la etiqueta. Los efectos de una dirección de memoria se han de tener en cuenta para los accesos posteriores.

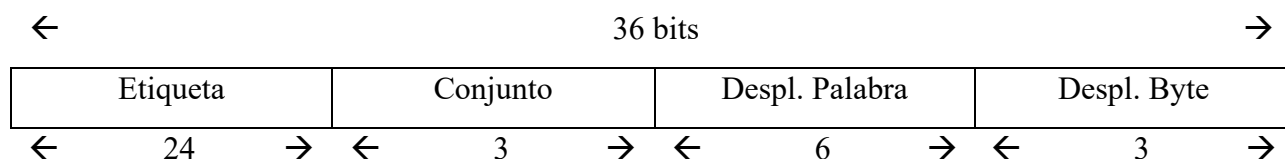
- Sabiendo que el tiempo de acierto de caché es de 2 ns, con una tasa de acierto del 90 %, calcular el tiempo medio de acceso a la memoria (AMAT). Tener en cuenta que la memoria principal está organizada con 16 bancos, cada uno de los cuales almacena una palabra. La latencia de memoria principal es de 30 ns por palabra (se pueden leer los 16 bancos a la vez). El tiempo para enviar una dirección a los bancos de la memoria y el tiempo para transferir palabra a palabra de la memoria a la caché a través del bus es de 4 ns. Los envíos de direcciones y de datos se pueden solapar todo lo posible para minimizar el tiempo total.

Solución:

a) Las direcciones de memoria son de 36 bits. El tamaño de palabra es de 64 bits = 8 bytes = 2^3 bytes, luego el desplazamiento de byte es de 3 bits. El tamaño de los bloques es de 64 palabras = 2^6 palabras, luego el desplazamiento de palabra es de 6 bits. Necesitamos calcular el número de conjuntos de la caché:

$$N.^{\circ} \text{ conjuntos} = \frac{8 \text{ kB}}{2^{\frac{\text{vías}}{\text{conjunto}}} \cdot 64^{\frac{\text{palabras}}{\text{vía}}} \cdot 8^{\frac{\text{bytes}}{\text{palabra}}}} = 8 \text{ conjuntos} = 2^3 \text{ conjuntos}$$

Por tanto, la dirección de memoria se descompone en los siguientes campos:



b) Si la caché está inicialmente vacía, su aspecto será como este:

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	0	1	...	...	0	0	...	...
1	0	1	...	...	0	0	...	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	1	...	...	0	0	...	...

Tal y como se pedía en el enunciado, no hemos dibujado la casilla correspondiente a los “datos”. Tampoco hemos rellenado las casillas de las etiquetas. Hemos puesto todos los bits de validez a 0, ya que no hay datos válidos en la caché. Por otro lado, hemos colocado todos los bits recientes de la vía 1 a 1 y todos los de la vía 0 a 0 para que, en caso de fallo, se lleve un bloque de datos a la vía 0.

Veamos qué efecto tienen las lecturas de las siguientes posiciones de memoria. En cada caso, habrá que descomponer la dirección en los campos del apartado anterior:

b.1) 0xABCDEF012 = 1010 1011 1100 1101 1110 1111 0000 0001 0010 = [1010 1011 1100 1101 1110 1111] [000] [00 0010] [010]

Etiqueta = 1010 1011 1100 1101 1110 1111 = 0xABCDEF

Conjunto = 000

DP = 00 0010

DB = 010

Miramos las dos vías del conjunto 000 de la caché y es un fallo, porque ambas están marcadas como V=0. La más reciente es la vía 1, con lo que se lleva el bloque de datos a la vía 0, que queda marcada con V=1 y R=1. La vía 1 queda con R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	0	0	...	...	1	1	0xABCDEF	...
1	0	1	...	...	0	0	...	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	1	...	...	0	0	...	...

b.2) 0x803223106 = 1000 0000 0011 0010 0010 0011 0001 0000 0110 = [1000 0000 0011 0010 0010 0011] [000] [10 0000] [110]

Etiqueta = 1000 0000 0011 0010 0010 0011 = 0x803223

Conjunto = 000

DP = 10 0000

DB = 110

Miramos las dos vías del conjunto 000 y es un fallo, ya que una de ellas está marcada con V=0 y la otra, aunque tiene V=1, no hay coincidencia de etiquetas. Como la más reciente es la vía 0, llevamos el bloque de datos a la vía 1, que pasa a tener V=1 y R=1. Asimismo, la vía 0 pasa a tener R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	1	1	0x803223	...	1	0	0xABCDEF	...
1	0	1	...	...	0	0	...	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	1	...	...	0	0	...	...

b.3) 0x250917E83 = 0010 0101 0000 1001 0001 0111 1110 1000 0011 = [0010 0101 0000 1001 0001 0111] [111] [01 0000] [011]

Etiqueta = 0010 0101 0000 1001 0001 0111 = 0x250917

Conjunto = 111

DP = 01 0000

DB = 011

Miramos en ambas vías del conjunto 111 y resulta ser un fallo, ya que ambas están marcadas con V=0. Como la más reciente es la vía 1, llevamos el bloque de datos a la vía 0, que marcamos con V=1 y R=1. Asimismo, la vía 1 queda con R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	1	1	0x803223	...	1	0	0xABCDEF	...
1	0	1	...	...	0	0	...	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	0	...	...	1	1	0x250917	...

b.4) 0x1234570ED = 0001 0010 0011 0100 0101 0111 0000 1110 1101 = [0001 0010 0011 0100 0101 0111] [000] [01 1101] [101]

Etiqueta = 0001 0010 0011 0100 0101 0111 = 0x123457

Conjunto = 000

DP = 01 1101

DB = 101

Miramos ambas vías del conjunto 000, que son válidas, pero no hay coincidencia de etiquetas, luego es un fallo. Como la más reciente es la vía 1, se lleva el bloque de datos a la vía 0, que se marca con R=1. Por su parte, la vía 1 se pone con R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	1	0	0x803223	...	1	1	0x123457	...
1	0	1	...	...	0	0	...	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	0	...	...	1	1	0x250917	...

b.5) 0xEDCEFD393 = 1110 1101 1100 1110 1111 1101 0011 1001 0011 = [1110 1101 1100 1110 1111 1101] [001] [11 0010] [011]

Etiqueta = 1110 1101 1100 1110 1111 1101 = 0xEDCEFD

Conjunto = 001

DP = 11 0010

DB = 011

Miramos ambas vías del conjunto 001 y vemos que están marcadas con V=0, luego es un fallo. Como la más reciente es la vía 1, llevamos el bloque de datos a la vía 0, que marcamos con V=1 y R=1. Asimismo, la vía 1 se pone con R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	1	0	0x803223	...	1	1	0x123457	...
1	0	0	...	...	1	1	0xEDCEFD	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	0	0	...	...	1	1	0x250917	...

b.6) 0x01630DF36 = 0000 0001 0110 0011 0000 1101 1111 0011 0110 = [0000 0001 0110 0011 0000 1101] [111] [10 0110] [110]

Etiqueta = 0000 0001 0110 0011 0000 1101 = 0x01630D

Conjunto = 111

DP = 10 0110

DB = 110

Miramos ambas vías del conjunto 111, pero una está marcada con V=0 y la otra sí tiene V=1, pero no hay coincidencia de etiquetas, luego es un fallo. Como la vía más reciente es la 0, llevamos el bloque de datos a la vía 1, que marcamos con V=1 y R=1. Asimismo, la vía 0 queda con R=0.

Conjunto	Vía 1				Vía 0			
	V	R	Etiqueta	Datos	V	R	Etiqueta	Datos
0	1	0	0x803223	...	1	1	0x123457	...
1	0	0	...	...	1	1	0xEDCEFD	...
2	0	1	...	...	0	0	...	...
3	0	1	...	...	0	0	...	...
4	0	1	...	...	0	0	...	...
5	0	1	...	...	0	0	...	...
6	0	1	...	...	0	0	...	...
7	1	1	0x01630D	...	1	0	0x250917	...

c) El tiempo medio de acceso a la memoria (AMAT) se calcula como:

$$AMAT = T_{\text{acierto}} + \% \text{ fallo} \cdot T_{\text{penalización}} = \% \text{ acierto} \cdot T_{\text{acierto}} + \% \text{ fallo} \cdot T_{\text{fallo}},$$

donde: $T_{\text{fallo}} = T_{\text{acierto}} + T_{\text{penalización}}$.

Según los datos: $T_{\text{acierto}} = 2 \text{ ns}$ y $\% \text{ acierto} = 0,90 \Rightarrow \% \text{ fallo} = 0,10$. Debemos calcular el tiempo de penalización por fallo, que corresponde al tiempo necesario para leer y llevar un bloque completo de la memoria principal a la vía correspondiente de la memoria caché.

Un bloque se compone de 64 palabras y cada palabra se compone de 8 bytes. Luego hay que leer en total 512 bytes en cada fallo de caché.

La memoria principal se compone de 16 bancos, cada uno con 1 palabra, con lo que habrá que acceder a ellos 4 veces para poder leer las 64 palabras del bloque.

Todas las medidas de tiempo están expresadas en ns:

4	30	4	...	4	30	4	...	4	30	4	...	4	30	4	...	4
Dir.				Dir.				Dir.				Dir.				
	Laten- cia	Pal. 1	...	Pal. 16	Laten- cia	Pal. 17	...	Pal. 32	Laten- cia	Pal. 33	...	Pal. 48	Laten- cia	Pal. 49	...	Pal. 64

Tras el tiempo de latencia de 30 ns, se envía por el bus de datos cada una de las 16 palabras leídas. Durante los 4 ns correspondientes a la última palabra del bloque, podemos enviar solapadamente la dirección del siguiente bloque de palabras. El tiempo total de penalización será:

$$T_{\text{penalización}} = 4 + 4 \cdot (30 + 16 \cdot 4) = 380 \text{ ns}$$

Por lo tanto, el *AMAT* es:

$$AMAT = 2 \text{ ns} + 0,1 \cdot 380 \text{ ns} = 40 \text{ ns}$$

9. [Ejercicio de examen] Sea un computador dotado con una jerarquía de memoria que incluye memoria caché y memoria principal. El ancho de palabra es de 32 bits y las direcciones de memoria son de 20 bits, con unidad direccionable el byte. Los detalles de la memoria caché son los siguientes:

- La caché es asociativa con 8 conjuntos de 2 vías cada uno.
- La transferencia entre la memoria principal y la caché se realiza mediante bloques de 4 palabras.
- La política de escritura es *write-back* y *write-allocate*.
- El algoritmo de reemplazo es LRU.
- Aparte de etiquetas y datos, la caché posee para cada vía un bit de validez (V), un bit de uso más reciente (R) y un bit de suciedad (D).
- El contenido parcial de la caché en un momento de dado es el siguiente:

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	1	1	0832	...	1	0	0	0DE6	...
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	0	0567	...	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	0	0	0	1FFF	...	1	1	1	0B6D	...

(etiquetas en hexadecimal, número de conjunto en decimal, por simplicidad no se muestran los datos)

Se pide:

- Indicar detalladamente los campos en los que se divide una dirección de memoria.
- Durante la ejecución de un programa se realiza la siguiente secuencia ordenada de accesos a memoria (direcciones en hexadecimal). Considerar que cada cambio en la caché afecta a los accesos siguientes. Indicar detalladamente si son aciertos o fallos de caché, cómo va cambiando la caché a lo largo del tiempo, si se actualiza la memoria principal y cuál es el estado final de la memoria caché.

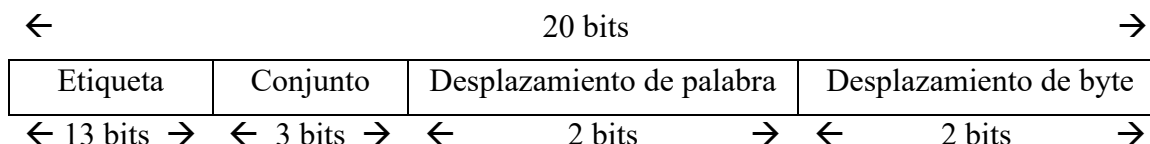
- 0x6F325 (lectura)
- 0xF09A2 (escritura)
- 0x2B3D8 (escritura)
- 0x8A074 (lectura)
- 0xC1AF0 (escritura)

- Si el tiempo de acceso a la memoria caché es de 2 ns, mientras que se requieren 50 ns para llevar un bloque entero de datos desde la memoria principal a la caché, calcular el tiempo medio de acceso a la memoria suponiendo una tasa de aciertos de caché del 85 %.

Solución:

a) La dirección de 20 bits se divide en los siguientes campos, de menor a mayor significación:

- Desplazamiento de byte: como las palabras son de 4 bytes ($=2^2$), este campo es de 2 bits.
- Desplazamiento de palabra: como los bloques son de 4 palabras ($=2^2$), este campo también tiene 2 bits.
- Número de conjunto: como hay 8 conjuntos ($=2^3$), este campo tiene 3 bits.
- Etiqueta: el resto de la dirección, en este caso, 13 bits.



b) A continuación, indicamos lo que ocurre con cada acceso a memoria.

b.1) $0x6F325 = 0110\ 1111\ 0011\ 0010\ 0101 = [0\ 1101\ 1110\ 0110] [010] [01] [01]$

Etiqueta = $0\ 1101\ 1110\ 0110 = 0DE6$

Conjunto = $010 = 2$

Operación = lectura

En el conjunto 2, encontramos esta etiqueta en la vía 0 con el bit de validez a 1. Luego es un ACIERTO. Marcamos la vía 0 como la más reciente y desmarcamos la vía 1.

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	0	1	0832	...	1	1	0	0DE6	dato
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	0	0567	...	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	0	0	0	1FFF	...	1	1	1	0B6D	...

b.2) $0xF09A2 = 1111\ 0000\ 1001\ 1010\ 0010 = [1\ 1110\ 0001\ 0011] [010] [00] [10]$

Etiqueta = $1\ 1110\ 0001\ 0011 = 1E13$

Conjunto = $010 = 2$

Operación = escritura

Buscamos esta etiqueta en el conjunto 2, pero no encontramos coincidencia. Luego es un FALLO. El dato más reciente se encuentra en la vía 0, luego traeremos un bloque de memoria principal a la vía 1, que ahora pasará a ser la más reciente. Pero primero, debemos ver que el bit de suciedad está activo, con lo que habrá que salvaguardar este bloque en memoria principal (por ser *write-back*) antes de traer el nuevo bloque a la vía 1 (por ser *write-allocate*). Tras traernos dicho bloque, lo modificamos en la caché, con lo que el bit de suciedad pasa a valer 1.

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	1	1	1E13	dato	1	0	0	0DE6	...
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	0	0567	...	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	0	0	0	1FFF	...	1	1	1	0B6D	...

b.3) $0x2B3D8 = 0010\ 1011\ 0011\ 1101\ 1000 = [0\ 0101\ 0110\ 0111]\ [101]\ [10]\ [00]$

Etiqueta = $0\ 0101\ 0110\ 0111 = 0567$

Conjunto = $101 = 5$

Operación = escritura

Vamos al conjunto 5 y encontramos coincidencia de etiquetas en la vía 1, que seguirá siendo la más reciente (R no cambia). Además, el dato es válido. Por tanto, es un ACIERTO. Al tratarse de una escritura, se modifica la caché, pero no la memoria principal (*write-back*). Para dar cuenta de este cambio, se activa el bit de suciedad. La memoria principal será actualizada cuando este bloque sea evacuado de la caché.

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	1	1	1E13	...	1	0	0	0DE6	...
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	1	0567	dato	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	0	0	0	1FFF	...	1	1	1	0B6D	...

b.4) $0x8A074 = 1000\ 1010\ 0000\ 0111\ 0100 = [1\ 0001\ 0100\ 0000]\ [111]\ [01]\ [00]$

Etiqueta = $1\ 0001\ 0100\ 0000 = 1140$

Conjunto = $111 = 7$

Operación = lectura

Vamos al conjunto 7, pero no hay coincidencia de etiquetas. Luego es un FALLO. A continuación, vemos que la vía más reciente es la 0, luego el bloque de memoria principal debe traerse a la vía 1, que será la más reciente ahora. Además, como el dato en la vía 1 es no válido, podemos copiar directamente el bloque de memoria principal. En ese caso, habrá que activar el bit de validez.

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	1	1	1E13	...	1	0	0	0DE6	...
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	1	0567	...	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	1	1	0	1140	dato	1	0	1	0B6D	...

b.5) $0xC1AF0 = 1100\ 0001\ 1010\ 1111\ 0000 = [1\ 1000\ 0011\ 0101]\ [111]\ [00]\ [00]$

Etiqueta = $1\ 1000\ 0011\ 0101 = 1835$

Conjunto = $111 = 7$

Operación = escritura

Buscamos esta etiqueta en el conjunto 7, pero no encontramos coincidencia. Luego es un FALLO. El dato más reciente se encuentra en la vía 1, luego traemos un bloque de memoria principal a la vía 0, que ahora pasará a ser la más reciente. Pero antes, debemos ver que el bit de suciedad está activo, con lo que habrá que salvaguardar este bloque en memoria principal (por ser *write-back*) antes de traer el nuevo bloque a la vía 0 (por ser *write-allocate*). Tras traernos dicho bloque, lo modificamos en la caché, con lo que el bit de suciedad pasa a valer 1.

Conjunto	Vía 1					Vía 0				
	V	R	D	Etiqueta	Datos	V	R	D	Etiqueta	Datos
0	0	0	0	1FFF	...	1	1	1	1A09	...
1	0	1	0	1FFF	...	0	0	0	1FFF	...
2	1	1	1	1E13	...	1	0	0	0DE6	...
3	1	0	0	1C00	...	1	1	0	039E	...
4	0	1	0	1FFF	...	0	0	0	1FFF	...
5	1	1	1	0567	...	1	0	0	1580	...
6	0	1	0	1FFF	...	0	0	0	1FFF	...
7	1	0	0	1140	...	1	1	1	1835	dato

c) El tiempo de acceso a la caché es el tiempo de acierto, mientras que el otro dato que nos dan es el tiempo de penalización.

$$\begin{aligned}
 AMAT &= T_{\text{acierto}} + \% \text{ fallo} \cdot T_{\text{penalización}} = \\
 &= 2 \text{ ns} + (1 - 0,85) \cdot 50 \text{ ns} = 9,5 \text{ ns}
 \end{aligned}$$

Otra manera de calcularlo es:

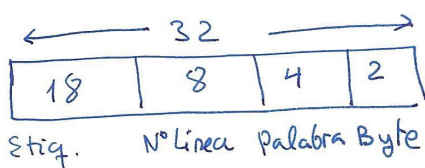
$$\begin{aligned}
 AMAT &= \% \text{ acierto} \cdot T_{\text{acierto}} + \% \text{ fallo} \cdot T_{\text{fallo}} = \\
 &= 0,85 \cdot 2 \text{ ns} + (1 - 0,85) \cdot (2 \text{ ns} + 50 \text{ ns}) = 9,5 \text{ ns}
 \end{aligned}$$

10. Calcular el número total de bits necesarios para una caché de 256 líneas y 16 palabras por bloque. Tanto las direcciones de memoria como el tamaño de palabra es 4 bytes, aunque la unidad direccionable es a nivel de byte. En el cálculo debe considerarse el espacio para guardar todos los datos, las etiquetas y los bits de validez.

Solución:

Tamaño total de la caché:

- 256 líneas $\rightarrow$ 8 bits
- 16 palabras por bloque $\rightarrow$ 4 bits
- Direcciones de memoria de 32 bits.
- Palabras de 4 bytes, unidad direccionable = byte.



$$\begin{aligned}
 \text{Tamaño total} &= 256 \text{ líneas} \times (1 + 18 + \\
 &\quad + 16 \frac{\text{palabras}}{\text{bloque}} \times 32 \frac{\text{bits}}{\text{palabra}}) = 135936 \text{ bits} \\
 &= 16992 \text{ bytes} \approx 16,6 \text{ KB}
 \end{aligned}$$

11. Sea un computador dotado de un sistema de jerarquía de memoria con memoria caché y principal y un ancho de palabra de 32 bits, con direcciones de memoria también de 32 bits.

a) La memoria caché:

- Tiene un tamaño de 4 kpalabras.
- Es asociativa por conjuntos de 2 vías.
- Cada bloque tiene 8 palabras.

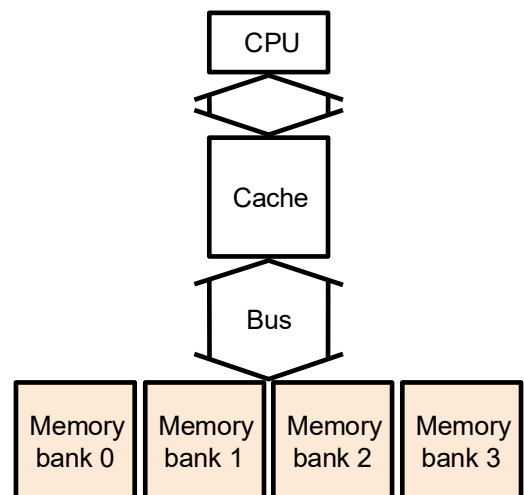
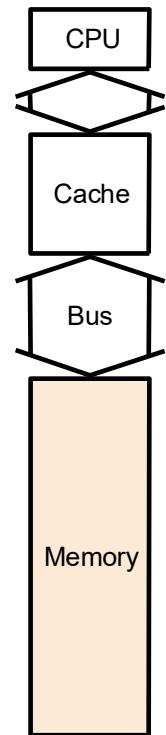
Indicar cuáles son los campos en los que se descompone una dirección de memoria de cara a la caché, cuántos bits se necesitan para cada uno de ellos y para qué se utilizan dichos campos.

- b) Supongamos que el ancho del bus es de una palabra. Por otra parte, cada vez que se quiere transferir un bloque de 8 palabras de memoria principal a caché:
- El procesador envía a la memoria la primera dirección del bloque, lo cual ocupa 1 ciclo de reloj.
 - La memoria tarda 15 ciclos en devolver la primera palabra del bloque al procesador.
 - El procesador envía a la memoria la siguiente dirección al mismo tiempo que la memoria transmite al procesador la palabra leída.
 - Una vez la memoria ha recibido la siguiente dirección de bloque y ha terminado de transmitir la palabra anterior, tarda otros 15 ciclos en enviar al procesador la palabra siguiente del bloque.
 - Los dos pasos anteriores se repiten hasta completar el envío del bloque completo.

Indicar cuántos ciclos consume un fallo de caché (penalización por fallo), es decir, cuánto tiempo se tarda en enviar un bloque completo desde memoria principal a caché.

- c) La tasa de fallos de la memoria caché es del 6 %, mientras que el tiempo de acierto consume un ciclo de reloj. ¿Cuál será el tiempo medio de acceso a memoria medido en ciclos y en segundos, teniendo en cuenta que un ciclo dura 2 ns?

- d) Manteniendo el ancho del bus en una palabra y organizando la memoria principal en forma entrelazada con 4 módulos de DRAM, cada vez que se envía una dirección se leen 4 palabras de un bloque simultáneamente, y se transmiten por el bus en serie, de modo que cuando la memoria transmite la última palabra del bloque, el procesador envía la siguiente dirección de palabra simultáneamente. ¿Cuántos ciclos duraría ahora un fallo de caché? ¿Cuál sería el tiempo medio de acceso a memoria medido en ciclos y en segundos? Utilizar los mismos valores de tasa de fallos y duración del ciclo del apartado c).



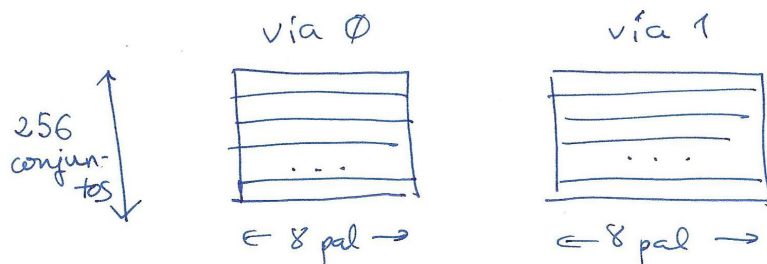
Solución:

- a) Ancho de palabra = 32 bits = 4 bytes
Direcciones de mem. = 32 bits.

Caché: $\begin{cases} 4 \text{ kpalabras} \\ \text{Asociativa por conjuntos de 2 vías.} \\ \text{1 bloque} = 8 \text{ palabras} = 2^3 \end{cases}$

Suponemos que la unidad direccionable es 1 byte.

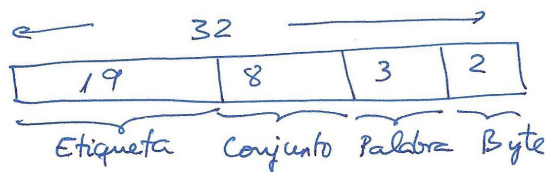
$$\Rightarrow \text{Nº conjuntos de caché: } 4 \text{ kpalabras} \times \frac{1 \text{ vía}}{8 \text{ palabras}} \times \frac{1 \text{ conj.}}{2 \text{ vías}} = 256 \text{ conjuntos}$$



1 palabra = 4 bytes = $2^2 \Rightarrow 2$ bits para el desplazamiento de byte.

1 bloque = 8 palabras = $2^3 \Rightarrow 3$ bits para el despl. de palabra o de bloque

256 conjuntos = $2^8 \Rightarrow 8$ bits para el conjunto (índice)



Direcc. de memoria.

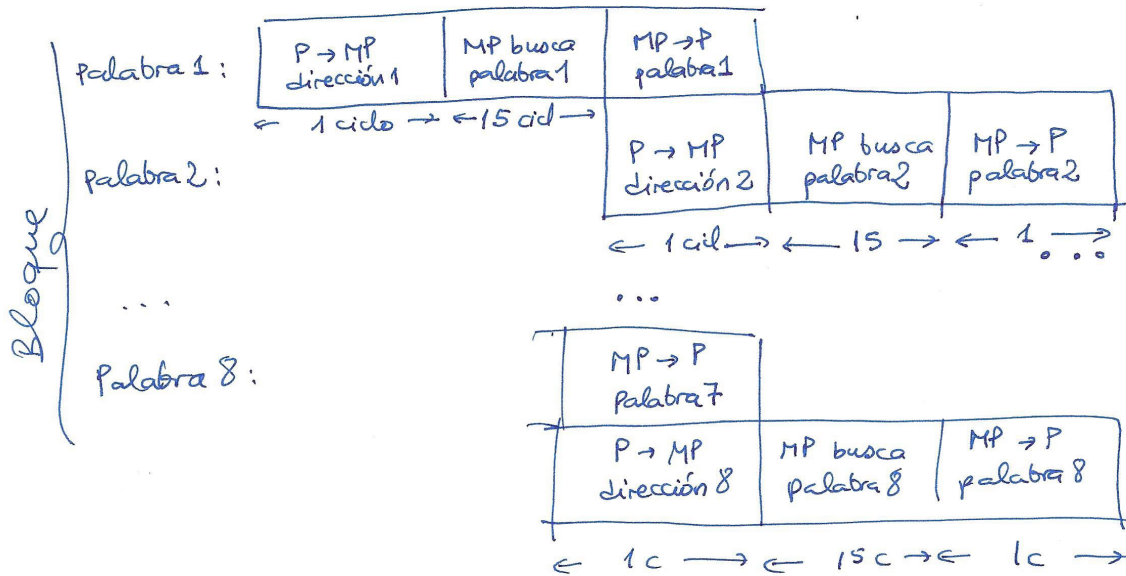
- b) Ancho de bus = 1 palabra.

Operación de transferencia de bloque (8 palabras):

¿Cuál es la penalización por fallo de caché?

- ① Proc $\rightarrow$ Mem: 1ª dirección de bloque $\rightarrow$ 1 ciclo.
- ② Mem: tarda 15 ciclos en devolver la primera palabra del bloque.
- ③ Proc $\rightarrow$ Mem: siguiente dirección } A la vez
Mem $\rightarrow$ Proc: palabra leída
- ④ Mem: Tarda otros 15 ciclos en enviar al proc. la palabra siguiente del bloque.
- ⑤ Esto se repite hasta enviar el bloque completo.

- * Ciclo 1: El procesador envía a memoria la dirección de la palabra 1 del bloque.
 - * Ciclos 2-16 (=15): la memoria busca la palabra.
 - * Ciclo 17 $\left\{ \begin{array}{l} \text{la memoria envía la palabra al proc.} \\ \text{El proc. envía la dirección de la siguiente palabra.} \end{array} \right.$
- etc.



$$\text{Total ciclos} = (1 + 15) \times 8 + 1 = 129 \text{ ciclos}$$

↳ Este es el tiempo necesario para llevar todo un bloque de MP al procesador $\equiv$ Tiempo de penalización por fallo de caché.

c) ¿AMAT?

$$1 \text{ ciclo} = 2 \text{ ns}$$

$$T_{\text{acceso}} = 1 \text{ ciclo}$$

$$T_{\text{penalización}} = 129 \text{ ciclos}$$

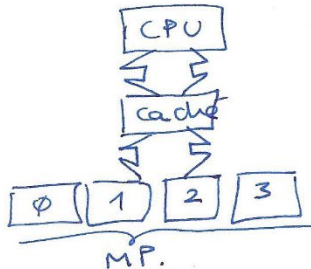
$$\% \text{ fallo} = 6\%$$

$$\begin{aligned} \Rightarrow \text{AMAT} &= T_{\text{acceso}} + \% \text{ fallo} \times T_{\text{penalización}} \\ &= 1 \text{ ciclo} + 0,06 \times 129 \text{ ciclos} = 8,74 \text{ ciclos} \\ &= 8,74 \text{ ciclos} \times \frac{2 \text{ ns}}{\text{ciclo}} = 17,48 \text{ ns} \end{aligned}$$

d) - Mantenemos el ancho del bus = 1 palabra = 32 bits

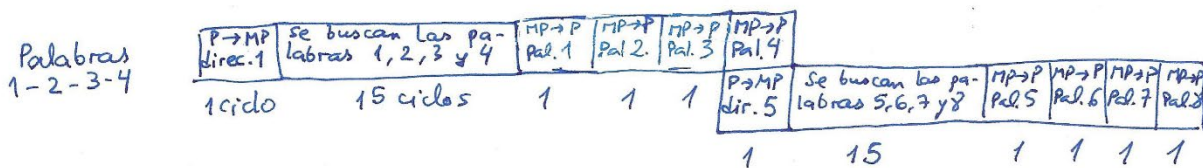
- Memoria principal entrelazada con 4 módulos DRAM

⇒ Se leen 4 palabras a la vez y se transmiten en serie.
Al enviar la última palabra el procesador envía la siguiente dirección.



¿Cuánto dura ahora un fallo de caché?

- Ahora el procesador envía la dirección de la primera palabra del bloque a la MP ↓
1 ciclo
- En paralelo cada módulo de MP lee una palabra (15 ciclos).
- Finalmente las 4 palabras leídas se envían en serie, cada una tarda 1 ciclo.
- Este último ciclo está solapado con el envío de la dirección de la quinta palabra del bloque.

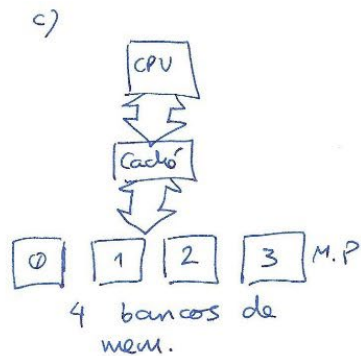
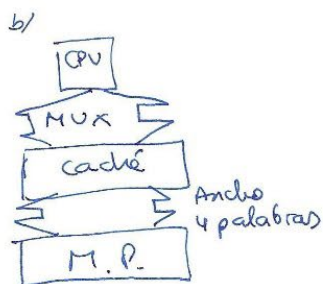
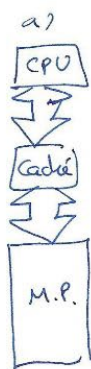


$$\Rightarrow T_{\text{penalización por fallo}} = (1 + 15 + 1 + 1 + 1) \times 2 + 1 = 39 \text{ ciclos}$$

$$\Rightarrow \text{AMAT} = 1 \text{ ciclo} + 0,06 \times 39 \text{ ciclos} = 3,34 \text{ ciclos} = 6,68 \text{ ns}$$

12. Considérese una jerarquía de memorias como la del ejercicio anterior. Se supondrá que el tamaño de bloque de la caché es de 16 palabras, que el ancho de la organización (b) de la figura es de 4 palabras y que el número de bancos de la organización (c) es de 4. Si la latencia de la memoria principal para un acceso nuevo es de 10 ciclos y el tiempo de transferencia es de 1 ciclo, ¿cuáles serán las penalizaciones por fallo en cada una de dichas organizaciones?

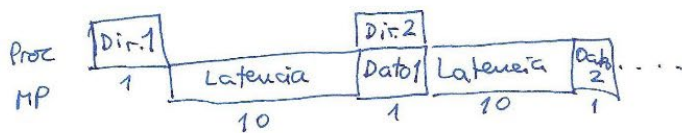
Solución:



Tamaño bloque = 16 palabras
 Latencia MP = 10 ciclos/palabra
 $T_{transferencia} = 1 \text{ ciclo/palabra}$

¿ Penalización por fallo? }

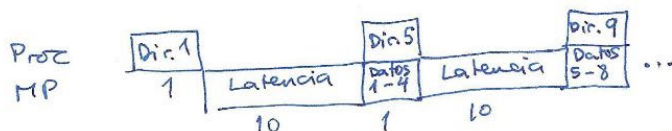
a) Hay 16 accesos a MP, cada uno de una palabra.



Penalización por fallo: $(10 + 1) \times 16 + 1 = 177 \text{ ciclos}$

$\uparrow$ $\uparrow$ $\uparrow$
 latencia transf. n° accesos

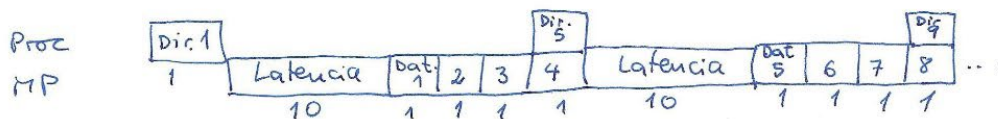
b) Hay 4 accesos a MP (4 palabras). Los datos leídos se transmiten en paralelo.



Penalización por fallo: $(10 + 1) \times 4 + 1 = 45 \text{ ciclos}$

$\uparrow$
 solo 4 accesos

c) Como el anterior, pero los datos se transmiten en serie.



Penalización por fallo: $(10 + 4) \times 4 + 1 = 57 \text{ ciclos}$

$\uparrow$
 N° accesos

13. Sea una máquina multicitelo que ejecuta un programa de computador con la siguiente frecuencia de ejecución de instrucciones:

Instrucción	Ciclos por instrucción (sin penalizaciones por fallo de caché)	Porcentaje sobre el total de instrucciones del programa
lw	5	29 %
sw	4	11 %
Aritmético-lógicas	4	40 %
Salto condicional	3	16 %
j	3	4 %

- Calcular el número medio de ciclos por instrucción (CPI) de dicho programa, suponiendo que no hay fallos de caché.
- Supongamos que el 2 % de los accesos a instrucciones y el 5 % de los accesos a datos producen fallo de caché. Si cada fallo implica una penalización de 40 ciclos, indicar cuál será el nuevo valor de CPI para el programa.
- Si el ciclo de reloj dura 2 ns, y dado que el tiempo de acierto es de 1 ciclo, indicar cuál es el tiempo medio de acceso a memoria medido en segundos.

Solución:

Máquina multicitelo:

Instrucción	CPI (sin fallos de caché)	% uso
lw	5	29%
sw	4	11%
Aritm.	4	40%
salto cond.	3	16%
j	3	4%

Acceso a datos en memoria = 40%
 (29% + 11% + 40% + 16% + 4% = 100%)

$CPI_{ideal} = \sum_i \text{ciclos}_i \times f_i$

a) CPI medio $\Rightarrow$ Hacemos media ponderada:

$$CPI_{ideal} = \frac{5 \times 0,29 + 4 \times 0,11 + 4 \times 0,40 + 3 \times 0,16 + 3 \times 0,04}{1} = 4,09 \text{ ciclos/instr.}$$

(ideal: sin fallos de caché).

b) Con fallos de caché:

$$CPI_{real} = CPI_{ideal} + CPI_{con fallos de caché}$$

los fallos de caché surgen de dos maneras:

- Por lecturas de las instrucciones: El 2% de ellas sufren una penalización de 40 ciclos.
 $\Rightarrow CPI_{fallo\ instr} = 0,02 \times 40 = 0,8 \text{ ciclos de penaliz.}$
- Por lecturas/escrituras de datos: Suponen el 40% de todas las instrucciones. Y de estos accesos el 5% tiene una penaliz. de 40 ciclos.
 $CPI_{fallo\ datos} = f_{datos} \times f_{fallo\ instr} \times P_{fallo}$

$$CPI_{\text{fallos datos}} = 0,40 \times 0,05 \times 40 = 0,8 \text{ ciclos de penaliz.}$$

Total: $CPI_{\text{real}} = 4,09 + 0,8 + 0,8 = 5,69 \text{ instrucciones/ciclo}$

c) Ciclo de reloj = 2 ns ¿AMAT?
Tacierto = 1 ciclo

$$AMAT = T_{\text{acierto}} + \frac{\% \text{ fallo}}{\text{conjunto para instrucciones y datos}} \times T_{\text{penalización}}$$

Tenemos:

- * Todas las instrucciones han de leerse de memoria. De cada 100 instrucciones, 2 dan fallo (2%).
- * Por otro lado, el 40% de las instrucciones leen o escriben datos de memoria, de las cuales el 5% dan fallo $\Rightarrow$ otros 2 fallos por cada 100 instrucciones.
- * Globalmente de cada 100 instrucciones de programa, se leen 100 instr. de memoria y se leen/escriben 40 datos de memoria $\Rightarrow$ total 4 fallos en 140 accesos a mem.

$$\Rightarrow \% \text{ fallo}_{\text{total}} = \frac{4}{140} = 0,0286 = 2,86 \%$$

$$\Rightarrow AMAT = 1 \text{ ciclo} \times 2 \frac{\text{ns}}{\text{ciclo}} + 0,0286 \times 40 \text{ ciclos} \times 2 \frac{\text{ns}}{\text{ciclo}} =$$

$$= 4,286 \text{ ns} = 4,286 \times 10^{-9} \text{ s} = 2,143 \text{ ciclos}$$

14. Al ejecutar un programa se tiene una tasa de fallos de instrucciones del 2 % y una tasa de fallos de datos del 4 %. Si la máquina tiene un CPI de 2 sin bloqueos de memoria y una penalización por fallo de 40 ciclos para todos los fallos, se pide:
- Determinar cuánto más rápida sería una máquina con una caché perfecta sin fallos si el programa tiene la siguiente proporción de instrucciones:
 - Repetir el cálculo anterior suponiendo ahora que se consigue reducir el CPI a 1 (posible con técnicas de segmentación), manteniendo el mismo ciclo de reloj.

Tipo instrucción	Proporción
lw (load word)	21 %
sw (store word)	12 %
lb (load byte)	1 %
sb (store byte)	1 %
lui (load upper immediate)	1 %
Aritmético-lógicas	39 %
Salto condicionales	15 %
j	10 %

Solución:

a) Las instrucciones que generan fallos por acceso a datos son lw, sw, lb y sb, que suponen el 35 % del total.

Ciclos de bloqueo por memoria = ciclos de bloqueo por lectura de instrucciones + ciclos de bloqueo por acceso a datos

Ciclos de bloqueo por lectura de instrucciones = $0,02 \times 40$ (fallos por instrucción $\times$ penalización de fallos)

Ciclos de bloqueo por acceso a datos (Instrucciones que acceden a datos: lw, sw, lb y sb, 35 % del total) = $0,35 \times 0,04 \times 40$ (fallos por instrucción $\times$ penalización de fallos)

Tasa de fallos conjunta = tasa de fallos por lectura de instrucciones + tasa de fallos por acceso a datos $\times$ tasa de instrucciones que acceden a datos = $0,02 + 0,04 \times 0,35 = 0,034$

Ciclos de bloqueo por memoria = $(0,02 + 0,35 \times 0,04) \times 40 = (0,02 + 0,014) \times 40 = 0,034 \times 40 = 1,36$
ciclos de bloqueo de memoria por instrucción

Entonces $CPI_real = CPI_ideal + \text{ciclos de bloqueo de memoria por instrucción} = 2 + 1,36 = 3,36$

Comparando con el caso sin bloqueos: $3,36 / 2 = 1,68$ veces más rápida la máquina con caché perfecta.

El porcentaje de tiempo de ejecución empleado en bloqueos de memoria sería el siguiente: $1,36 / 3,36 = 0,41 = 41 \%$

Recordemos la **Ley de Amdahl**: la posible mejora del rendimiento está limitada por la proporción en que se utiliza esa prestación mejorada.

b) Reducción del CPI de 2 a 1 sin cambiar la frecuencia de reloj (posible con segmentación, es decir, solapando varias instrucciones que se encontrarían en distinta fase de ejecución).

CPI ideal (sin fallos de caché) = 1

CPI real (con fallos de caché) = 1 + 1,36 = 2,36

Sería 2,36 veces más rápido con caché perfecta.

% tiempo de ejecución empleado en bloqueos de memoria habría crecido:

$$\text{de } \frac{1,36}{3,36} = 41 \% \quad \text{a} \quad \frac{1,36}{2,36} = 58 \%$$

15. [Ejercicio de examen] Sea un computador dotado con una jerarquía de memoria que incluye memoria caché y memoria principal. El ancho de palabra es de 8 bits y las direcciones de memoria son de 16 bits. Los detalles de la memoria caché son los siguientes:

- La caché es de correspondencia directa con 4 líneas.
- La transferencia entre la memoria principal y la caché se realiza mediante bloques de 8 palabras.
- La política de escritura es *write-back* y *write-allocate*.
- Aparte de etiquetas y datos, la caché posee para cada línea un bit de validez (V) y un bit de suciedad (D).
- El contenido de la caché en un momento de dado es el siguiente:

Línea	V	D	Etiquetas	Datos
0	1	1	0x7A0	...
1	1	0	0x47D	...
2	1	1	0x1CB	...
3	1	1	0x65E	...

(por simplicidad no se muestran los datos)

Se pide:

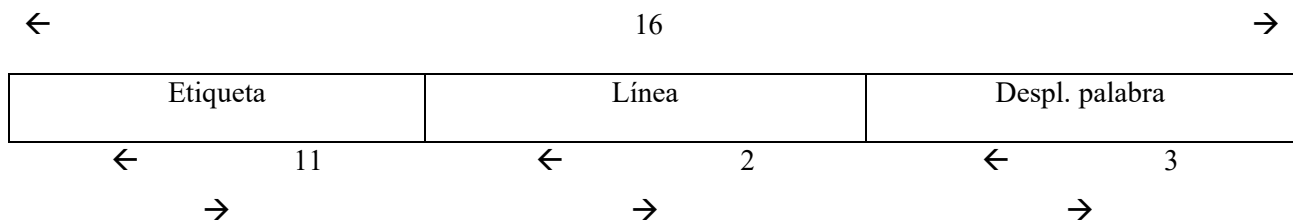
- Indicar detalladamente los **campos** en los que se divide una dirección de memoria.
- Durante la ejecución de un programa se realiza la siguiente secuencia ordenada de accesos a memoria (direcciones en hexadecimal). Considerar que cada cambio en la caché afecta a los accesos siguientes. Indicar detalladamente si son aciertos o fallos de caché, explicar detalladamente cómo va cambiando la caché a lo largo del tiempo, si se actualiza la memoria principal y cuál es el estado final de la memoria caché.

- b.1)** 0x8FAC (escritura)
- b.2)** 0x524F (lectura)
- b.3)** 0x0BAE (lectura)
- b.4)** 0x8FA8 (escritura)

- c) Si el tiempo de acceso a la memoria caché es de **4 ns**, mientras que se requieren **60 ns** para llevar un bloque entero de datos desde la memoria principal a la caché, calcular cuál debería ser el porcentaje de aciertos mínimo para que el tiempo medio de acceso a la memoria no excediera los **8 ns**. Redondear el porcentaje a un decimal.

Solución:

- a) La dirección de memoria se descompone de la siguiente manera:



Donde:

- El desplazamiento de byte es 0 bits porque la palabra es de tamaño 8 bits = 1 byte = 2^0 bytes.
- El desplazamiento de palabra es de 3 bits porque los bloques son de 8 palabras = 2^3 palabras.
- Hay $4 = 2^2$ líneas.
- El resto de los bits hasta completar 16 corresponden a la etiqueta, es decir, $16 - 2 - 3 - 0 = 11$ bits. Por ello, la etiqueta se expresa en hexadecimal con 3 dígitos, donde el más significativo solo puede ser un número de 0 a 7, ambos incluidos.

b.1) 0x8FAC (escritura)

Dirección = 0x8FAC = 1000 1111 1010 1100 = 100 0111 1101 // 01 // 100

Etiqueta = 100 0111 1101 = 0x47D

Línea = 01 = 1

DP = 100 = 4

Vamos a la línea 1 y vemos que V vale 1 y que hay coincidencia de etiquetas. Es un **ACIERTO** de caché. Al ser *write-back*, actualizamos directamente el bloque de datos de la caché y se activa el bit de suciedad.

El contenido de la caché es ahora:

Línea	V	D	Etiquetas	Datos
0	1	1	0x7A0	...
1	1	1	0x47D	Bloque actualizado
2	1	1	0x1CB	...
3	1	1	0x65E	...

b.2) 0x524F (lectura)

Dirección = 0x524F = 0101 0010 0100 1111 = 010 1001 0010 // 01 // 111

Etiqueta = 010 1001 0010 = 0x292

Línea = 01 = 1

DP = 111 = 7

Vamos a la línea 1 y vemos que no hay coincidencia de etiquetas. Es un FALLO de caché. Como el bit D está activado, llevamos en primer lugar el bloque de caché a la memoria principal. A continuación, leemos en memoria principal el bloque nuevo y lo guardamos en la línea 1. El bit V sigue siendo 1. El bit D pasa a ser 0 porque ahora hay coincidencia entre el bloque guardado en caché y el de memoria principal. La etiqueta es también actualizada al nuevo valor. El contenido de la caché es ahora:

Línea	V	D	Etiquetas	Datos
0	1	1	0x7A0	...
1	1	0	0x292	Bloque actualizado
2	1	1	0x1CB	...
3	1	1	0x65E	...

b.3) 0x0BAE (lectura)

Dirección = 0x0BAE = 0000 1011 1010 1110 = 000 0101 1101 // 01 // 110

Etiqueta = 000 0101 1101 = 0x05D

Línea = 01 = 1

DP = 110 = 6

Vamos de nuevo a la línea 1 y vemos que no hay coincidencia de etiquetas. Es un FALLO de caché. Como el bit D no está activado, podemos traer directamente el nuevo bloque de memoria principal a la caché. El bit V sigue siendo 1. El bit D sigue siendo 0. La etiqueta es también actualizada al nuevo valor.

El contenido de la caché es ahora:

Línea	V	D	Etiquetas	Datos
0	1	1	0x7A0	...
1	1	0	0x05D	Bloque actualizado
2	1	1	0x1CB	...
3	1	1	0x65E	...

b.4) 0x8FA8 (escritura)

Dirección = 0x8FA8 = 1000 1111 1010 1000 = 100 0111 1101 // 01 // 000

Etiqueta = 100 0111 1101 = 0x47D

Línea = 01 = 1

DP = 000 = 0

Vamos de nuevo a la línea 1 y vemos que no hay coincidencia de etiquetas. En esta ocasión se trata de la etiqueta con la que obtuvimos acierto en el apartado b.1, pero ya hemos borrado ese bloque. Es otra vez un FALLO de caché. Como el bit D no está activado, podemos traer directamente el nuevo bloque de memoria principal a la caché. El bit V sigue siendo 1. El bit D pasa a valer 1, porque el bloque de datos es actualizado en la caché. La etiqueta es también actualizada al nuevo valor. El contenido de la caché es ahora:

Línea	V	D	Etiquetas	Datos
0	1	1	0x7A0	...
1	1	1	0x47D	Bloque actualizado
2	1	1	0x1CB	...
3	1	1	0x65E	...

Los cuatro accesos a caché han sido a la misma línea, aunque los bloques se están machacando unos a otros. Se trata de un “fallo por conflicto”.

c) El tiempo medio de acceso a la memoria (*AMAT*, por sus siglas en inglés) se puede escribir como:

$$AMAT = \text{Tiempo de acierto} + \text{Tasa de fallo} \cdot \text{Tiempo de penalización por fallo}$$

En este caso:

- Tiempo de acierto = 4 ns
- Tiempo de penalización por fallo = 60 ns
- Tiempo total de fallo = $4 + 60 = 64$ ns
- Tasa de fallo = $1 - \text{Tasa de acierto}$ (en tanto por uno)
- $AMAT \leq 8$ ns

Calculemos para qué porcentaje de aciertos se obtiene exactamente $AMAT = 8$ ns.

$$8 \text{ ns} = 4 \text{ ns} + (1 - \text{Tasa de acierto}) \cdot 60 \text{ ns}$$

$$8 = 4 + 60 - 60 \text{ Tasa de acierto}$$

$$60 \text{ Tasa de acierto} = 56$$

$$\text{Tasa de acierto} = \frac{56}{60} = 0.9\bar{3}$$

O lo que es lo mismo, redondeando a un decimal, un porcentaje de aciertos del 93,3 %. Si el porcentaje de aciertos es MAYOR QUE este valor, se reducirá el tiempo medio de acceso a la memoria por debajo de los 4 ns.

16. [Ejercicio de examen] Sea un computador dotado con una jerarquía de memoria que incluye memoria caché y memoria principal. El ancho de palabra es de 16 bits y las direcciones de memoria son de 16 bits, con unidad direccionable el byte. Los detalles de la memoria caché son los siguientes:

- La caché es asociativa con 2 conjuntos de 4 vías cada uno.
- La transferencia entre la memoria principal y la caché se realiza mediante bloques de 8 palabras.
- La política de escritura es *write-back* y *write-allocate*.
- El algoritmo de reemplazo es LRU.
- Aparte de etiquetas y datos, la caché posee para cada vía: un bit de validez (V); dos bits de uso reciente (R_1R_0), donde la vía más reciente del conjunto es la 00, la segunda más reciente es la 01, la tercera más reciente es la 10 y la menos reciente de todas es la 11; y además un bit de suciedad (D).

El contenido parcial de la caché en un momento de dado es el siguiente:

Conj	Vía 3					Vía 2					Vía 1					Vía 0				
	V	R ₁ R ₀	D	Etiqu	Dat	V	R ₁ R ₀	D	Etiqu	Dat	V	R ₁ R ₀	D	Etiqu	Dat	V	R ₁ R ₀	D	Etiqu	Dat
0	1	10	1	1A8	...	1	00	0	65D	...	1	01	0	507	...	1	11	1	485	...
1	1	01	0	7FF	...	1	11	0	3EC	...	1	00	1	479	...	1	10	1	32B	...

(etiquetas en hexadecimal, por simplicidad no se muestran los datos)

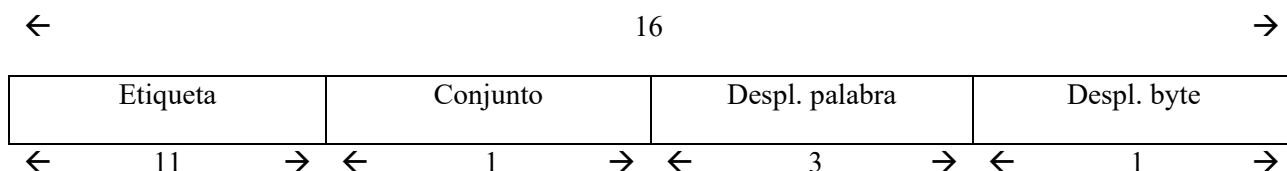
Se pide:

- Realizar la descomposición en campos de la dirección de memoria, indicando claramente el tamaño en bits y el nombre de cada campo.
- Durante la ejecución de un programa se realiza la siguiente secuencia ordenada de accesos a memoria (direcciones en hexadecimal). Considerar que cada cambio en la caché afecta a los accesos siguientes. Indicar **detalladamente** si son aciertos o fallos de caché, explicar cómo va cambiando la caché a lo largo del tiempo (rellenar las tablas para mostrar el estado de la caché tras cada apartado) y si se actualiza la memoria principal. Expresar las etiquetas en hexadecimal.
 - 0x55E5 (escritura)
 - 0xCBAE (escritura)
 - 0x3274 (escritura)
- Tras múltiples accesos a la memoria, se ha medido que la tasa de aciertos de la caché es del 84 %. Si el tiempo de acierto es de 9 ns, mientras que la penalización por fallo es 62,5 ns, calcular el tiempo medio de acceso a la memoria.

Solución:

- Las direcciones son de 16 bits y se descomponen de la siguiente manera:

- Desplazamiento de byte: 1 bit, porque las palabras son de 16 bits = 2 B = 2¹ B.
- Desplazamiento de palabra: 3 bits, porque los bloques son de 8 palabras = 2³ palabras.
- Conjunto: 1 bit, porque hay 2 conjuntos = 2¹ conjuntos.
- Etiqueta: 16 – 1 – 3 – 1 = 11 bits.



b) Pasemos ahora a realizar las escrituras en las direcciones de memoria que indica el resultado.

b.1) 0x55E5

Paso 1: realizar la descomposición en campos de dicha dirección de memoria.

$$0x55E5 = 0101\ 0101\ 1110\ 0101 = 010\ 1010\ 1111\ //\ 0\ //\ 010\ //\ 1$$

$$\text{Etiqueta} = 010\ 1010\ 1111 = 0x2AF$$

$$\text{Conjunto} = 0$$

$$DP = 010 = 2$$

$$DB = 1$$

Paso 2: consultamos todas las vías del conjunto 0 y detectamos no hay coincidencia de etiquetas, con lo que es un FALLO DE CACHE. En la inspección de las vías, encontramos que la menos reciente es la 0 ($R_1R_0 = 11$) y es la que será sustituida.

Paso 3: puesto que el bit D está activado, leemos el bloque almacenado en la vía 0 del conjunto 0 y lo escribimos en la memoria.

Paso 4: a continuación, por ser *write-allocate*, leemos el nuevo bloque en la memoria y lo escribimos en la vía 0 del conjunto 0, de forma que sustituye al antiguo bloque.

Paso 5: por ser *write-back*, realizamos la escritura que nos pide el enunciado en el bloque almacenado en la caché. Al contener un bloque más reciente comparado con la memoria principal, dejamos activado el bit D.

Paso 6: actualizamos todos los bits de uso reciente del conjunto 0. La vía 0 pasa a ser la más reciente (00). Los bits de uso reciente de las otras vías ven incrementado su valor en una unidad.

La caché queda de la siguiente manera:

Conj	Vía 3					Vía 2					Vía 1					Vía 0				
	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat
0	1	11	1	1A8	...	1	01	0	65D	...	1	10	0	507	...	1	00	1	2AF	Nuevo bloque
1	1	01	0	7FF	...	1	11	0	3EC	...	1	00	1	479	...	1	10	1	32B	...

b.2) 0xCBAE

Paso 1: realizar la descomposición en campos de dicha dirección de memoria.

$$0xCBAE = 1100\ 1011\ 1010\ 1110 = 110\ 0101\ 1101\ //\ 0\ //\ 111\ //\ 0$$

$$\text{Etiqueta} = 110\ 0101\ 1101 = 0x65D$$

$$\text{Conjunto} = 0$$

$$DP = 111 = 7$$

$$DB = 0$$

Paso 2: consultamos todas las vías del conjunto 0 y detectamos coincidencia de etiquetas en la vía 2, donde además $V = 1$, con lo que es un ACIERTO DE CACHE.

Paso 3: realizamos la escritura solo en la caché por ser *write-back* y activamos el bit D.

Paso 4: La vía 2 pasará a ser la más reciente (00). La vía 0 verá incrementado en una unidad sus bits de uso reciente. Para las otras vías, no cambian.

La caché queda de la siguiente manera:

Conj	Vía 3					Vía 2					Vía 1					Vía 0				
	V	R ₁ R ₀	D	Etq	Dat	V	R ₁ R ₀	D	Etq	Dat	V	R ₁ R ₀	D	Etq	Dat	V	R ₁ R ₀	D	Etq	Dat
0	1	11	1	1A8	...	1	00	1	65D	Nuevo bloque	1	10	0	507	...	1	01	1	2AF	...
1	1	01	0	7FF	...	1	11	0	3EC	...	1	00	1	479	...	1	10	1	32B	...

b.3) 0x3274

Paso 1: realizar la descomposición en campos de dicha dirección de memoria.

$$0x3274 = 0011\ 0010\ 0111\ 0100 = 001\ 1001\ 0011\ //\ 1\ //\ 010\ //\ 0$$

$$\text{Etiqueta} = 001\ 1001\ 0011 = 0x193$$

$$\text{Conjunto} = 1$$

$$DP = 010 = 2$$

$$DB = 0$$

Paso 2: consultamos todas las vías del conjunto 1 y detectamos no hay coincidencia de etiquetas, con lo que es un FALLO DE CACHÉ. En la inspección de las vías, encontramos que la menos reciente es la 2 ($R_1R_0 = 11$) y es la que será sustituida.

Paso 3: puesto que el bit D está desactivado, podemos sobrescribir directamente el bloque contenido en la vía 2. Por tanto, por ser *write-allocate*, leemos el nuevo bloque en la memoria y lo escribimos en la vía 2 del conjunto 1, de forma que sustituye al antiguo bloque.

Paso 5: por ser *write-back*, realizamos la escritura que nos pide el enunciado en el bloque almacenado en la caché. Al contener un bloque más reciente comparado con la memoria principal, activamos el bit D.

Paso 6: actualizamos todos los bits de uso reciente del conjunto 1. La vía 2 pasa a ser la más reciente (00). Los bits de uso reciente de las otras vías ven incrementado su valor en una unidad.

La caché queda de la siguiente manera:

Conj	Vía 3					Vía 2					Vía 1					Vía 0				
	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat	V	R_1R_0	D	Etiqu	Dat
0	1	11	1	1A8	...	1	00	1	65D	...	1	10	0	507	...	1	01	1	2AF	...
1	1	10	0	7FF	...	1	00	1	193	Nuevo bloque	1	01	1	479	...	1	11	1	32B	...

c) El tiempo medio de acceso a la memoria (*AMAT*, por sus siglas en inglés) se calcula como:

$$AMAT = \text{Tiempo}_{\text{acierto}} + \text{Penalización}_{\text{fallo}} \times \text{Tasa}_{\text{fallo}} = 9 \text{ ns} + 62,5 \text{ ns} \times (1 - 0,84) = 19 \text{ ns}$$

17. [Ejercicio de examen] Sea un computador dotado con una jerarquía de memoria que incluye memoria caché y memoria principal. El ancho de palabra es de 16 bits y las direcciones de memoria también son de 16 bits, con unidad direccionable el byte. Los detalles de la memoria caché son los siguientes:

- La caché es totalmente asociativa con 4 líneas.
- La transferencia entre la memoria principal y la caché se realiza mediante bloques de 4 palabras.
- La política de escritura es *write-back* y *write-allocate*.
- Aparte de etiquetas y datos, la caché posee para cada línea un bit de validez (V), un bit de suciedad (D) y dos bits de uso reciente (R_1R_0 , donde **la línea más reciente es la 11**, la segunda más reciente es la 10, la tercera más reciente es la 01 y la menos reciente de todas es la 00).
- El contenido parcial de la caché en un momento de dado es el siguiente:

Línea	Etiqueta	V	D	R ₁ R ₀	Bloque de datos
0	1FAC	0	0	00	...
1	1E58	1	0	10	...
2	0A61	1	1	11	...
3	1C7D	1	1	01	...

(etiquetas en hexadecimal, número de línea en decimal, por simplicidad no se muestran los datos)

Se pide:

- Indicar detalladamente los campos en los que se divide una dirección de memoria.
- Durante la ejecución de un programa se realiza la siguiente secuencia ordenada de accesos a memoria (direcciones en hexadecimal). Considerar que cada cambio en la caché afecta a los accesos siguientes. Indicar detalladamente si son aciertos o fallos de caché, explicar detalladamente cómo va cambiando la caché a lo largo del tiempo, el rango de direcciones de memoria principal modificadas (expresadas en hexadecimal) y cuál es el estado final de la memoria caché.

b.1) 0x810A (lectura)

b.2) 0x59B3 (escritura)

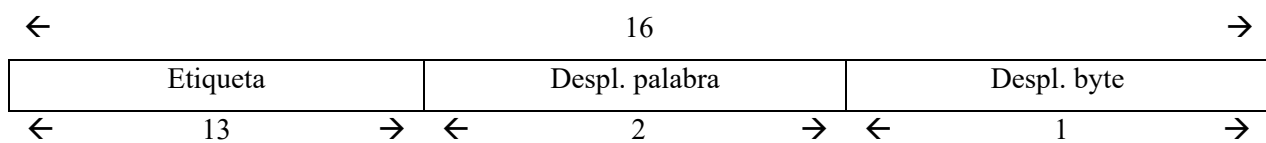
b.2) 0xF2C6 (escritura)

IMPORTANTE: rellenar las tablas de la siguiente página con los estados de la caché.

- Si el tiempo de acceso a la memoria caché es de 7 ns, mientras que se requieren 50 ns para llevar un bloque entero de datos desde la memoria principal a la caché, calcular el tiempo medio de acceso a la memoria suponiendo una tasa de aciertos de caché del 84 %.

Solución:

- Las direcciones de memoria se descomponen de la siguiente manera:



- Las palabras son de 16 bits = 2 bytes = 2^1 bytes, con lo que el desplazamiento de byte es 1 bit.
- Los bloques son de 4 palabras = 2^2 palabras, con lo que el desplazamiento de palabra es 2 bits.
- El resto de los bits de la dirección son para la etiqueta. En este caso, son $16 - 1 - 2 = 13$ bits.

b.1) 0x810A (lectura)

Descomponemos la dirección de memoria en campos:

Dirección = 0x810A = 1000 0001 0000 1010 = 1 0000 0010 0001 // 01 // 0

Etiqueta = 1 0000 0010 0001 = 0x1021

DP = 01 = 1

DB = 0

Miramos en todas las líneas para ver si hay coincidencia de etiquetas. No la hay, con lo que se trata de un FALLO DE CACHE.

La línea menos reciente es la 0. Al tener $V = 0$, podemos traer directamente el bloque de memoria principal a dicha línea y activamos V . Al ser una lectura, mantenemos $D = 0$. Debemos actualizar también los bits de uso reciente de todas las líneas. La línea 0 pasa a ser la más reciente (11). Para las demás líneas, se resta 1 al valor que tienen en R_1R_0 .

Línea	Etiqueta	V	D	R_1R_0	Bloque de datos
0	1021	1	0	11	datos
1	1E58	1	0	01	...
2	0A61	1	1	10	...
3	1C7D	1	1	00	...

b.2) 0x59B3 (escritura)

Descomponemos la dirección de memoria en campos:

Dirección = 0x59B3 = 0101 1001 1011 0011 = 0 1011 0011 0110 // 01 // 1

Etiqueta = 0 1011 0011 0110 = 0x0B36

DP = 01 = 1

DB = 1

Miramos en todas las líneas para ver si hay coincidencia de etiquetas. No la hay, con lo que se trata de un FALLO DE CACHE.

La línea menos reciente es la 3. Se ve que $V = 1$, con lo que el bloque de datos existente en la caché es válido. Como $D = 1$, es un bloque que se ha modificado después de haberse traído a la caché y necesita copiarse en memoria principal antes de ser sustituido por el nuevo bloque.

La salvaguarda del bloque se realiza entre las siguientes direcciones de memoria:

Etiqueta antigua = 0x1C7D = 1 1100 0111 1101

Dirección de inicio = 1 1100 0111 1101 // 00 // 0 = 1110 0011 1110 1000 = 0xE3E8

Dirección de fin = 1 1100 0111 1101 // 11 // 1 = 1110 0011 1110 1111 = 0xE3EF

Una vez hecha esta salvaguarda en memoria principal, traemos el nuevo bloque hasta la línea 3. Al tratarse de una escritura, la modificación se hace en la copia de la caché, con lo que D se mantiene a 1. Finalmente, se actualizan todos los bits de uso reciente.

Línea	Etiqueta	V	D	R ₁ R ₀	Bloque de datos
0	1021	1	0	10	...
1	1E58	1	0	00	...
2	0A61	1	1	01	...
3	0B36	1	1	11	datos

b.3) 0xF2C6 (escritura)

Descomponemos la dirección de memoria en campos:

Dirección = 0xF2C6 = 1111 0010 1100 0110 = 1 1110 0101 1000 // 11 // 0

Etiqueta = 1 1110 0101 1000 = 0x1E58

DP = 11 = 3

DB = 0

Miramos en todas las líneas para ver si hay coincidencia de etiquetas. La hay en la línea 1 y además V = 1, por tanto, se trata de un ACIERTO DE CACHE.

La escritura se realiza en la caché, con lo que debemos hacer que D = 1. También actualizamos todos los bits de uso reciente.

Línea	Etiqueta	V	D	R ₁ R ₀	Bloque de datos
0	1021	1	0	01	...
1	1E58	1	1	11	datos
2	0A61	1	1	00	...
3	0B36	1	1	10	...

c) La fórmula del tiempo medio de acceso a la memoria (*AMAT*) es:

$$AMAT = \text{Tiempo}_{\text{acierto}} + \text{Tasa}_{\text{fallo}} \cdot \text{Penalización}_{\text{fallo}}$$

Donde:

- $\text{Tiempo}_{\text{acierto}} = 7 \text{ ns}$
- $\text{Penalización}_{\text{fallo}} = 50 \text{ ns}$
- $\text{Tasa}_{\text{fallo}} = 1 - 0,84 = 0,16$

Por tanto:

$$AMAT = 7 \text{ ns} + 0,16 \cdot 50 \text{ ns} = 15 \text{ ns}$$